

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

—o0o—



NGHIÊN CỨU TỐI ƯU LỊCH TRÌNH
CHO BÀI TOÁN CHIA SẺ CHUYỂN ĐI
KHÓA LUẬN TỐT NGHIỆP HỆ CHÍNH QUY
Ngành: Công nghệ thông tin

Sinh viên : Phạm Xuân Bách
Lớp : QH-2021-I/CQ-I-IT1
Cán bộ hướng dẫn : TS Vũ Thị Hồng Nhạn

HÀ NỘI - 2024

TÓM TẮT

Tóm tắt: Hiện nay, Việt Nam đang là một nước thuộc nhóm nước đang phát triển có tốc độ tăng trưởng kinh tế rất nhanh. Đi kèm theo đó là nguy cơ khi cơ sở hạ tầng không đáp ứng đủ lưu lượng giao thông tại các thành phố lớn, gây ra tình trạng tắc nghẽn giao thông và ô nhiễm không khí nghiêm trọng. Nguyên nhân chính là sự gia tăng nhanh chóng của các phương tiện cá nhân. Nhiều người sử dụng ô tô/xe máy chỉ để di chuyển một mình, khiến nhiều chỗ ngồi trong xe bị bỏ trống, dẫn đến lãng phí không gian và việc sử dụng tài nguyên không hiệu quả. Từ đó, mô hình đi chung xe ra đời, có tiềm năng giải quyết được vấn đề tắc đường nhờ gộp chung những yêu cầu của nhiều người để phục vụ trong một phương tiện duy nhất. Khóa luận này sẽ tập trung nâng cao chất lượng của dịch vụ đi chung xe nhờ tối ưu thuật toán tạo lịch trình của dịch vụ từ nhận định rằng các mô hình đặt xe trực tuyến từ phương tiện cá nhân hiện tại có một lượng dữ liệu về lịch sử yêu cầu đi xe rất lớn được tích lũy qua nhiều năm ở nhiều khu vực khác nhau. Những dữ liệu này được đưa vào các mô hình học máy để có thể đưa ra dự đoán nhu cầu đi xe với mỗi khung thời gian - địa điểm. Dựa trên những dữ liệu đã biết trước này, hệ thống đi chung thông minh sẽ tận dụng và lập lịch cho phương tiện tham gia giao thông. Hệ thống sẽ tạo ra những nhóm đi chung với mỗi taxi dựa trên các yêu cầu dự đoán có thể sẽ xuất hiện từ dữ liệu lịch sử, sau đó gán các yêu cầu đi chung xe trực tuyến của người dùng vào các nhóm đi chung đã lập trước, từ đó có thể tạo ra các nhóm đi chung xe tốn ít chi phí và phục vụ được nhiều yêu cầu hơn. Kết quả của việc phát triển này hứa hẹn triển khai được một hệ thống có thể đưa vào sử dụng thực tiễn với mục tiêu cải thiện được hệ thống đi chung xe cũ, giúp mọi người dễ dàng tiếp cận được với dịch vụ đi chung xe với mức chi phí thấp hơn và trải nghiệm tốt hơn, từ đó làm giảm vấn đề ùn tắc giao thông và vấn đề ô nhiễm tại các thành phố.

Từ khóa: Dịch vụ đi chung xe, yêu cầu đi chung xe trực tuyến, dự đoán nhu cầu đi chung xe, tối ưu lịch trình đi chung.

LỜI CẢM ƠN

Để có thể hoàn thành đề tài khoá luận một cách chu đáo, bên cạnh sự nỗ lực cố gắng của bản thân, tôi còn nhận được sự hướng dẫn nhiệt tình của quý thầy cô Trường Đại học Công nghệ, cũng như sự động viên ủng hộ của gia đình và bạn bè trong suốt thời gian học tập nghiên cứu và thực hiện đề tài khoá luận tốt nghiệp.

Tôi xin gửi lời cảm ơn sâu sắc đến cô giáo TS. Vũ Thị Hồng Nhạn, là người đã trực tiếp hướng dẫn, định hướng và giúp đỡ tôi trong suốt quá trình làm đề tài khoá luận tốt nghiệp này. Tôi cũng xin gửi lời cảm ơn chân thành tới toàn thể thầy cô giáo của Trường Đại học Công nghệ - Đại học Quốc Gia Hà Nội đã mang sự nhiệt tình, tâm huyết và niềm đam mê cháy bỏng cũng như những kinh nghiệm quý báu của mình để dạy dỗ, chỉ bảo, hướng dẫn tôi trong suốt thời gian học tập tại trường.

Tôi cũng xin cảm ơn gia đình, bạn bè đã là chỗ dựa vững chắc, luôn ủng hộ, động viên tôi trong suốt quá trình học tập tại Trường Đại học Công nghệ - Đại học Quốc gia Hà Nội. Do kiến thức và kinh nghiệm nghiên cứu còn hạn chế nên khoá luận tốt nghiệp không thể tránh khỏi sai sót, mong các thầy, cô góp ý để tôi có thể hoàn thiện khoá luận tốt hơn.

LỜI CAM ĐOAN

Tôi xin cam đoan khóa luận tốt nghiệp do tôi tự thực hiện dưới sự hướng dẫn của cô Vũ Thị Hồng Nhạn. Mọi thông tin tham khảo được dùng trong khóa luận đều sẽ được trích dẫn một cách đầy đủ và hợp lệ.

Tôi cam đoan không sao chép từ bất kì tài liệu, công trình nghiên cứu của người khác mà không ghi rõ ràng tài liệu tham khảo. Tôi xin chịu hoàn toàn trách nhiệm và mọi hình thức kỷ luật theo quy định của nhà trường dành cho lời cam đoan của mình.

Hà Nội, ngày tháng năm 2024

Sinh viên Phạm Xuân Bách

Mục lục

1	GIỚI THIỆU	7
1.1	Đặt vấn đề	7
1.2	Nội dung khóa luận	9
1.3	Cấu trúc khóa luận	10
2	CƠ SỞ LÝ THUYẾT	11
2.1	Lịch sử nghiên cứu với các hệ thống đặt xe	11
2.2	Các giải pháp hiện nay	11
3	THIẾT KẾ HỆ THỐNG VÀ THUẬT TOÁN LẬP LỊCH	14
3.1	Tổng quan về hệ thống	14
3.2	Mô hình hệ thống	18
3.3	Xây dựng ô lưới không gian để biểu diễn vị trí	19
3.4	Xây dựng lưới và xử lý dữ liệu điểm di chuyển	19
3.5	Cơ sở dữ liệu	20
3.6	Lập lịch và tối ưu lịch trình cho taxi	21
4	MÔI TRƯỜNG THỰC NGHIỆM	42
4.1	Nền tảng môi trường	42
4.2	Xây dựng dữ liệu	42
4.3	Xây dựng dữ liệu bản đồ lưới	44
4.4	Đánh giá hiệu năng thuật toán	45
4.5	Danh sách API	48
4.6	Xây dựng giao diện website của ban quản lý	50
5	KẾT LUẬN	54
5.1	Tổng kết	54
5.2	Định hướng phát triển	54
	Phụ lục	57
A	CHI TIẾT CÁC BẢNG Ở CƠ SỞ DỮ LIỆU	57

Danh sách hình vẽ

3.1	Biểu đồ use case tổng quát của hệ thống	15
3.2	Mô hình hệ thống cho chức năng tìm kiếm chuyến đi chung trực tuyến . .	18
3.3	Ví dụ minh họa ánh xạ một điểm lên bản đồ lưới	19
3.4	Mô hình cơ sở dữ liệu	21
3.5	Các trường hợp khi chen o_r và d_r	26
3.6	[Ví dụ] Đồ thị mạng lưới giao thông	36
4.1	Dữ liệu các lịch trình hướng dẫn taxi di chuyển tới các nơi có nhiều yêu cầu	42
4.2	Dữ liệu lịch trình của một nhóm đi chung	43
4.3	Dữ liệu yêu cầu đi chung xe trực tuyến	43
4.4	Dữ liệu yêu cầu đi chung xe trực tuyến	44
4.5	Số lượng yêu cầu được phục vụ	46
4.6	Số lượng taxi cần sử dụng để phục vụ yêu cầu đi chung xe trực tuyến . . .	47
4.7	Chi phí mỗi một yêu cầu được phục vụ	47
4.8	Thời gian tổng để phục vụ 200 yêu cầu đi chung xe (đơn vị: s)	48
4.9	Giao diện đăng nhập của ban quản lý	50
4.10	Giao diện quản lý chính	51
4.11	Giao diện nhóm các taxi và lịch trình hiện tại trên bản đồ	51
4.12	Giao diện danh sách taxi đang hoạt động	52
4.13	Giao diện chi tiết taxi	52
4.14	Giao diện tạo taxi mới	53
4.15	Giao diện tạo tài khoản cho tài xế	53

Danh sách bảng

3.1	Danh sách tác nhân tham gia hệ thống	15
3.2	Chức năng xem lịch trình hiện tại	16
3.3	Chức năng xem lịch trình các taxi theo thời gian thực	16
3.4	Chức năng xem thống kê của hệ thống	17
3.5	Chức năng tạo tài khoản tài xế	17
3.6	Bảng thông tin các yêu cầu động mới xuất hiện r_1-r_3	37
3.7	Bảng khởi tạo các mảng cho ví dụ 3.6.5	38
A.1	Bảng admin	57
A.2	Bảng passenger	57
A.3	Bảng driver	58
A.4	Bảng taxi	58

A.5	Bảng OTP	58
A.6	Bảng route	59
A.7	Bảng grid	59
A.8	Bảng edge	60
A.9	Bảng request_ride	60
A.10	Bảng predicted_request	61
A.11	Bảng type_status	61
A.12	Bảng group_frequent	62
A.13	Bảng schedule	62
A.14	Bảng guidance_schedule	63
A.15	Bảng type_location	63
A.16	Bảng type_schedule	64

Chương 1. GIỚI THIỆU

1.1 Đặt vấn đề

Ngày nay, tắc nghẽn giao thông đã trở thành một vấn đề trên toàn thế giới, đặc biệt là ở các khu vực đô thị. Theo báo cáo của UBND Thành phố Hà Nội, diện tích đất sử dụng cho giao thông, đường xá, bãi đỗ xe trên tổng diện tích đất xây dựng đô thị mới chỉ đạt 12,13% trong khi mức quy hoạch tối thiểu là 20-26%. [1]. Cùng với đó là tốc độ phát triển số lượng ô tô/xe máy một cách chóng mặt, gấp đôi với con số hơn 1 triệu xe ô tô sau gần 5 năm từ 2018 [2]. Điều này gây áp lực tới hạ tầng giao thông cũng như làm trầm trọng hơn việc tắc đường ở đô thị. Theo thống kê với dữ liệu từ Viện Giao thông vận tải Texas A&M [3], mỗi một người Mỹ lãng phí khoảng 35-37 tiếng di chuyển đến chỗ làm và về nhà trong một năm vì tắc đường, kết quả là công việc cũng phát triển chậm đi. Tắc nghẽn giao thông không chỉ làm mất thời gian mà còn gây ra thiệt hại rất lớn cho chính phủ của các nước. Theo sổ giao thông vận tải thành phố Hà Nội, tình trạng giao thông tồi tệ khiến địa phương này thiệt hại khoảng 1 - 1.2 tỷ USD mỗi năm¹. Trong khi đó con số này ở thành phố Hồ Chí Minh lên đến 6 tỷ USD mỗi năm, theo sổ giao thông vận tải thành phố Hồ Chí Minh².

Đã có rất nhiều nghiên cứu tìm ra các giải pháp đối với việc phát triển hạ tầng và tiềm lực của riêng từng khu vực trên toàn cầu để giải quyết vấn đề này. Tuy nhiên, nghiên cứu có thể tìm ra được giải pháp đã khó mà để thực hiện chúng cần rất nhiều thời gian, đặc biệt đây là vấn đề liên quan với giao thông, cơ sở vật chất của đất nước. Vì thế, biện pháp khả thi nhất có thể thực hiện được ngay mà không cần phải đợi hạ tầng giao thông đáp ứng đủ lưu lượng lưu thông là tối ưu hóa cách di chuyển của người dân, mà một trong các cách là đẩy mạnh dịch vụ đi chung xe. Cơ sở của giải pháp đó là hầu hết những khung giờ tắc đường vào thời điểm học sinh đi học, nhân viên đi làm và những lúc tan học tan làm. Những vị trí đến và đi của người dân cũng thường là những tòa nhà lớn, trường học. Nếu chỉ tính theo khung giờ và khu vực, dường như những điểm đến/thời gian này là cố định. Với giải pháp đi xe buýt hiện tại, chia sẻ xe có lợi hơn về trải nghiệm người sử dụng khi thoải mái ngồi, hơn nữa là tốc độ cũng sẽ nhanh hơn dù giá cả cao hơn. Thời gian đi chung có tăng hơn so với đi một mình và cố định về tuyến đường hơn nhưng đi chung xe sẽ phần nào giải quyết được việc tắc đường, gián tiếp làm cho thời gian di chuyển ít đi. Chi phí di chuyển cũng được chia sẻ nên thấp hơn so với di chuyển một mình. Giải pháp này đem lại lợi ích cực kỳ lớn cho xã hội khi giảm thiểu được lượng phương tiện lưu thông

¹<https://baomoi.com/ha-noi-thiet-hai-do-un-tac-giao-thong-len-den-1-2-ty-usd-nam-c-47605828.epi>

²<https://daidoanket.vn/un-tac-giao-thong-gay-lang-phi-hang-ty-usd-10227018.html>

cùng một thời điểm, giảm lượng khí thải thải ra môi trường và nhiên liệu sử dụng tổng thể. Hơn nữa đây là một giải pháp không yêu cầu cải thiện/xây mới các hình thức giao thông, cần ít vốn - thời gian để đầu tư hệ thống hơn các hình thức tàu điện, tàu cao tốc.

Tuy nhiên, giải pháp này cần một hệ thống có thể đưa ra quyết định ghép các yêu cầu đi xe chung thời gian thực với đặc điểm nhanh, chính xác, tối ưu được chi phí. Hệ thống còn phải xử lý tốt khi có nhiều yêu cầu mà không đủ lượng tài xế phục vụ dẫn tới hủy yêu cầu đi chung của người dùng. Vì việc này ảnh hưởng rất lớn tới trải nghiệm khi sử dụng ứng dụng, làm giảm doanh thu cũng như giảm tỉ lệ quay lại sử dụng dịch vụ. Từ đó, giải pháp không chỉ cần phải giải quyết việc chen yêu cầu đi chung xe vào lịch trình hiện tại của tài xế hiệu quả, mà còn phải phục vụ nhiều yêu cầu nhất có thể.

Với những xe đi chung đang di chuyển trên hành trình của chúng, làm thế nào có thể chen được yêu cầu mới là điểm đón và điểm trả người dùng vào lịch trình hiện tại. Hệ thống phải lấy đầu vào là yêu cầu và đầu ra là lịch trình mới được sắp xếp theo thời gian thực sao cho chi phí di chuyển tăng thêm là ít nhất. Có rất nhiều nghiên cứu đã và đang tìm cách để giải quyết vấn đề này: [4] tối ưu thời gian khi chen một cặp điểm đón, điểm trả thông qua việc sử dụng bản đồ lưới và một ma trận tĩnh chứa khoảng cách cũng như thời gian giữa các điểm; [5] nghiên cứu về tần suất đi lại dựa trên lịch sử di chuyển của người dùng qua đó khai phá ra tuyến đường thường xuyên của người dùng; [6] nghiên cứu về ghép nhóm đi chung dựa trên các tuyến đường đã có sẵn của người dùng. Một số vấn đề cũng như hạn chế của các nghiên cứu trên đó chính là chỉ mới giải quyết được bài toán đi chung xe cho các hành khách có tuyến đường thường xuyên, chưa giải quyết được bài toán cho người mới sử dụng hệ thống hay phương pháp xử lý chung cho toàn bộ hệ thống. Hơn nữa, hiện nay lượng dữ liệu về đặt xe của các công ty đã vận hành được nhiều năm là rất lớn. Nhiều nghiên cứu mới về một dịch vụ có thể đưa ra dự đoán trước về yêu cầu đặt xe theo khung không gian - địa điểm [7] và độ chính xác đạt tới 80% [8]. Dựa trên những dữ liệu có sẵn đó, ta có thể điều hướng các xe taxi đi tới những khu vực nhiều yêu cầu trước, làm giảm thời gian di chuyển khi nhận yêu cầu mới và đưa được taxi vào những khu vực có thể có nhiều yêu cầu cần phục vụ theo kết quả dự đoán.

Khóa luận này tiến hành cài đặt, phát triển và đánh giá thuật toán xử lý chen thêm một yêu cầu vào lịch trình của một xe trong nghiên cứu [9] cũng như thuật toán Prophet xử lý các yêu cầu được dự đoán trước để hướng dẫn các taxi di chuyển tới nơi có yêu cầu đi xe [10]. Sau đó khóa luận đề xuất một thuật toán mới có tên Prune Prophet với ý tưởng là tối ưu thời gian tìm kiếm vị trí chen cặp điểm của yêu cầu. Cùng với đó, khóa luận này sẽ thiết kế, phân tích một hệ thống quản lý taxi đi chung thực tế.

Việc tối ưu hóa lập lịch cho bài toán đi chung xe đòi hỏi sự kết hợp giữa các yếu tố

toán học, hệ thống và quản lý, và có tiềm năng tác động đáng kể đến cuộc sống hàng ngày của cộng đồng cũng như môi trường. Nghiên cứu này có thể đóng góp lớn vào việc phát triển các hệ thống giao thông công cộng thông minh và bền vững hơn, giúp giảm thiểu ô nhiễm không khí và tạo điều kiện thuận lợi cho việc di chuyển của mọi người. Bằng cách áp dụng các phương pháp như thuật toán Prophet, chúng ta có thể dự đoán và xử lý hiệu quả hơn các biến động trong nhu cầu đi lại, từ đó nâng cao hiệu suất của dịch vụ đi chung xe.

1.2 Nội dung khóa luận

Khoá luận này nghiên cứu bài toán lập lịch trình cho hệ thống cung cấp dịch vụ đi chung xe. Giả sử có một số xác định các taxi đang hoạt động và một tập yêu cầu đi xe được gửi từ người dùng, hệ thống cần xử lý và gán từng yêu cầu đi xe đó cho từng taxi sao cho tối ưu nhất về chi phí và thời gian di chuyển. Để gán một yêu cầu vào một taxi, hay nói cách khác là chèn yêu cầu mới vào lịch trình di chuyển hiện tại của taxi, hệ thống phải tính toán và đưa ra vị trí chèn tối ưu nhất.

Trong khóa luận này, tôi hướng đến trường hợp hệ thống đã có bộ dữ liệu dự đoán yêu cầu đặt xe của người dùng trong một khoảng khung giờ, địa điểm nhất định. Dữ liệu này sẽ được tiền xử lý và tạo ra các lịch trình hướng dẫn - lịch trình mà taxi đi theo chỉ dẫn của hệ thống tới những nơi được dự đoán sẽ có yêu cầu. Khi những khách hàng mới muốn di chuyển từ điểm đón O sang điểm trả D bất kỳ, các thuật toán được cài đặt trong khóa luận sẽ tìm ra taxi với lịch trình thỏa mãn nhất với yêu cầu khách hàng. Mục tiêu của giải pháp là tối ưu thời gian xử lý, tối đa hoá số lượng yêu cầu được phục vụ và tối thiểu chi phí phục vụ tất cả các yêu cầu.

Khoá luận này khảo sát và đánh giá hai cách tiếp cận:

- **Phương pháp chèn thô (Plain):** với mỗi yêu cầu mới, thuật toán chèn cặp điểm đón, trả vào trong lịch trình, tối ưu thời gian và quãng đường đi thêm.

Theo cách tiếp cận này, tôi triển khai hai thuật toán là Basic Plain và Linear Plain có độ phức tạp thuật toán lần lượt là $O(n^3)$ và $O(n)$. Với từng thuật toán, thời gian chạy và chi phí để phục vụ hàng loạt yêu cầu đi xe chung được phân tích chi tiết theo sự thay đổi của các thông số trong yêu cầu.

Cách tiếp cận thô này có nhược điểm là khi có yêu cầu mới, hệ thống sẽ tìm các xe ở gần đó, thêm yêu cầu này vào một trong các xe đó. Nếu các xe đều không ở gần hoặc quá nhiều xe ở một khu vực nhỏ không có nhiều nhu cầu đi xe, hệ thống có thể không phục vụ được tất cả các yêu cầu sau này nếu chúng xuất hiện ở những nơi khác. Vì vậy, khi hệ thống có ít taxi hoạt động, các taxi đó nên được hướng đến

những nơi có tần suất đi xe chung nhiều. Thuật toán xử lý trường hợp này được trình bày ở phần sau.

- **Phương pháp chen với dự đoán(Prophet):** với bộ dữ liệu về các yêu cầu được dự đoán với thời gian xuất hiện của từng yêu cầu, thuật toán tạo ra lịch trình hướng dẫn taxi di chuyển tới những địa điểm "nóng" - dự đoán là có nhiều yêu cầu. Các taxi khi tới những địa điểm này sẽ đợi cho tới khi yêu cầu dự đoán xuất hiện thật sự. Thuật toán phải thực hiện chen cặp điểm đón, trả vào lịch trình hướng dẫn đó của taxi sao cho phục vụ được nhiều yêu cầu nhất với lượng taxi có hạn.

Theo cách tiếp cận này, tôi triển khai hai thuật toán là Prophet và Prune Prophet được cải tiến từ thuật toán Linear Plain cùng có độ phức tạp thuật toán là $O(n)$. Với từng thuật toán, thời gian chạy, chi phí phục vụ, số lượng yêu cầu được phục vụ thời gian thực được phân tích chi tiết theo sự thay đổi của các thông số trong yêu cầu.

Cuối cùng, một hệ thống thử nghiệm sẽ được phát triển trong đó áp dụng các thuật toán đã nghiên cứu. Hệ thống gồm hai phần hệ là quản trị và cung cấp dịch vụ tới người dùng và phần hệ người dùng tương tác với hệ thống. Trong giới hạn của khoá luận này, tôi tập trung phát triển phần hệ quản trị và cung cấp dịch vụ.

1.3 Cấu trúc khóa luận

Khóa luận gồm năm chương như sau:

Chương 1: Giới thiệu: đặt ra vấn đề đang xuất hiện trong thực tế, các hạn chế của giải pháp hiện tại và lý do chọn đề tài.

Chương 2: Cơ sở lý thuyết: giới thiệu các nghiên cứu liên quan trước đó cùng định hướng. Các nghiên cứu về cách giải quyết bài toán đã có và hiện tại.

Chương 3: Thiết kế hệ thống và thuật toán lập lịch: Phân tích chi tiết các thuật toán đề ra và quá trình thiết kế hệ thống logic của ứng dụng.

Chương 4: Xây dựng thực nghiệm: giới thiệu sản phẩm thực nghiệm, trình bày quá trình thực nghiệm, xây dựng, cách vận hành và đánh giá hệ thống.

Chương 5: Kết luận: đưa ra các kết quả thu được và định hướng phát triển sắp tới.

Chương 2. CƠ SỞ LÝ THUYẾT

Việc đi chung xe không phải là mới. Nhưng hiện nay, nó thường được tài xế và khách hàng tự thương lượng với nhau để đi chung xe hoặc cũng có một số hệ thống tự động ghép nhóm đi chung nhưng chưa thực sự hiệu quả. Một số giải pháp đã được đưa ra là: đi chung xe định kỳ và đi chung xe trực tuyến. Những giải pháp này chỉ có đi chung xe định kỳ là tận dụng được dữ liệu có sẵn về yêu cầu gọi xe của người dùng, nhưng cũng chỉ tiền xử lý và đưa ra các nhóm đi chung được cố định, không thêm được các yêu cầu trực tuyến vào lịch trình. Các giải pháp này cũng chưa tối ưu hoàn toàn chi phí di chuyển, vận hành của dịch vụ gọi xe.

2.1 Lịch sử nghiên cứu với các hệ thống đặt xe

Việc nghiên cứu về dịch vụ đặt xe di chuyển đã được công bố từ những năm 70 của thế kỷ trước [11] cũng như được đánh giá bởi các nhà khoa học thuộc các lĩnh vực về cơ sở dữ liệu, khai phá dữ liệu hay giao thông. Từ đó, bài toán đặt xe được nghiên cứu nhiều hơn với từng trường hợp nhỏ, nhưng ảnh hưởng lớn tới kết quả chung như tình trạng giao thông, thời tiết, cách thực hiện chia sẻ chuyến đi,

Một đặc điểm quan trọng của hệ thống đặt xe đó là đặt xe động hay đặt xe tĩnh. Dịch vụ đặt xe tĩnh là dịch vụ mà quãng đường, yêu cầu, tài xế đã được biết trước như dịch vụ đi chung xe định kỳ. Ngược lại, đặt xe động là khi yêu cầu di chuyển xuất hiện không theo quy luật nào cả, ngẫu nhiên và phải được phục vụ trong thời gian ngắn. Những hệ thống ứng dụng gọi xe phổ biến hiện nay như Grab, Be, XanhSM đều cung cấp dịch vụ gọi xe động cho người dùng, giúp người dùng di chuyển theo yêu cầu ngay lập tức.

Các dịch vụ gọi xe này đều cần giải quyết các bài toán tối ưu quãng đường, thời gian, phục vụ được nhiều yêu cầu đặt xe nhất có thể. Quãng đường đi ít hơn sẽ giảm chi phí cũng như khí thải ra môi trường; càng nhiều yêu cầu di chuyển của khách hàng được hoàn thành càng làm tăng doanh số cũng như uy tín của dịch vụ.

2.2 Các giải pháp hiện nay

Để giải quyết các bài toán trên, một số nghiên cứu đã được đề xuất như: giảm thiểu quãng đường đi và cố gắng phục vụ nhiều yêu cầu nhất [12]; giảm thời gian chờ đợi được phục vụ của người dùng [13]; giảm thời gian hoàn thành yêu cầu của tài xế [14]; tối ưu cho từng bài toán đi chung, ...

2.2.1 Đi chung xe định kỳ

Nòng cốt của dịch vụ đi chung xe định kỳ là đáp ứng nhu cầu đi lại thường nhật, điển hình là hành trình giữa nhà và nơi làm việc, được lặp lại hầu hết các ngày trong tuần (từ thứ Hai đến thứ Sáu). Để hiện thực hóa mô hình này, hai thách thức lớn cần được khắc phục. Trước hết, việc khai thác dữ liệu di chuyển của người dùng để xác định các tuyến đường được sử dụng thường xuyên là vô cùng quan trọng. Nhiều phương pháp đã được đề xuất, trong đó thuật toán FP-Growth được biết đến như một giải pháp hiệu quả [15]. Tuy nhiên, việc tìm ra các tuyến đường thường xuyên chỉ là bước đầu tiên. Thách thức tiếp theo, không kém phần quan trọng, là việc ghép nối những người dùng có tuyến đường tương đồng để tạo thành các nhóm đi chung xe. Đây là một bài toán tối ưu hóa phức tạp. Một số thuật toán đã được nghiên cứu và ứng dụng cho bài toán này, bao gồm AGP, VG-Growth và aVG-Growth [16]. Sự kết hợp hiệu quả giữa việc xác định tuyến đường thường xuyên và việc ghép nhóm người dùng sẽ quyết định sự thành công của dịch vụ đi chung xe định kỳ.

Tuy nhiên dịch vụ đi chung xe định kỳ mới chỉ phục vụ được cho những hành khách có tuyến đường thường xuyên mà chưa phục vụ được cho những khách hàng có yêu cầu đi chung xe trực tuyến.

2.2.2 Đi chung xe trực tuyến

Dịch vụ đi chung xe trực tuyến đã từng xuất hiện trên thị trường Việt Nam. Ví dụ điển hình là vào 09/05/2017 Grab - ứng dụng đặt xe công nghệ hàng đầu Đông Nam Á đã cho ra mắt dịch vụ GrabShare tại thành phố Hồ Chí Minh ³. Mục đích của GrabShare là ghép nhóm các yêu cầu đi chung xe trực tuyến để giảm thiểu ùn tắc giao thông, giảm tới 30% chi phí so với GrabCar thông thường cho hành khách đồng thời tăng thu nhập cho các tài xế. Thuật toán kết hợp được sử dụng trong GrabShare sẽ tính toán và xác định việc ghép xe đi chung dựa trên các yếu tố như: tài xế có sẵn gần nhất, thời gian di chuyển, các tuyến đường trùng nhau, quãng đường đi vòng và điều kiện giao thông ở thời điểm đặt xe.

Tuy nhiên vấn đề gặp phải ở đây là việc ghép nhóm các yêu cầu đi chung xe trực tuyến là rất khó. Có rất nhiều trường hợp ghép được nhóm đi chung nhưng quãng đường đi của một người trong số các hành khách đi chung là rất nhỏ, nên chi phí giảm thiểu cho hành khách còn lại là rất ít. Dẫn đến việc chi phí giảm thiểu không đáng kể so với việc hành khách tốn thời gian và quãng đường di chuyển lại tăng thêm do có thêm hành khách.

³<https://www.grab.com/vn/press/business/grab-ra-mat-dich-vu-di-chung-xe-grabshare-de-gop-phan-giam-ap-luc-giao-thong-tai-cac-thi/>

Công trình [17] đã đưa ra một cái nhìn tổng quan về những thách thức và cơ hội của chia sẻ chuyến đi động, đặt nền móng cho các cải tiến trong thiết kế hệ thống sau này. Nhiều giải pháp được đề xuất qua các nghiên cứu dựa trên cơ chế chen - một yêu cầu mới được chen vào lịch trình đã có của tài xế. Trong bài toán thực tế, giải pháp phải đưa ra quyết định có tài xế nào sẵn sàng, phù hợp để phục vụ yêu cầu mới này ngay khi hệ thống nhận được yêu cầu hay không? Hệ thống thường phải thực hiện các bước sau đây kể từ khi nhận yêu cầu của khách hàng: tìm tài xế ở gần đó, xem xét những tài xế đó có thể phục vụ yêu cầu với quãng đường tối thiểu, kiểm tra các điều kiện khác như số lượng chỗ ngồi, ... Các hệ thống như T-Share [4] tìm ra một danh sách các tài xế có thể phù hợp qua chỉ mục bản đồ, những tài xế gần đó sẽ được đưa vào danh sách, sau đó chen yêu cầu mới vào tài xế có quãng đường đi thêm là ít nhất. Nghiên cứu [18] sử dụng chỉ mục loại cây thay đổi để lưu trữ tất cả các tuyến đường có thể và dùng thuật toán chen để đưa yêu cầu mới vào lịch trình. Các nghiên cứu này đều có đặc điểm chung là khi chen yêu cầu mới vào lịch trình cũ, thời gian chạy đều rất lớn $O(n^2)$ cho tới $O(n^3)$, vì chỉ tối ưu được một giai đoạn nhỏ trong quá trình tính toán.

Mặc dù đã đạt được nhiều tiến bộ, các thách thức đang tồn tại đòi hỏi nghiên cứu sâu hơn, nhất là về thời gian xử lý một yêu cầu mới - thứ liên quan trực tiếp tới trải nghiệm người dùng. Khoá luận trước đây [19] đã thực hiện cài đặt và đánh giá thuật toán Linear Plain với bài toán tối ưu thời gian chen lịch trình. Mặc dù kết quả đạt được rất tốt khi đã giảm độ phức tạp thời gian chạy từ $O(n^3)$ xuống $O(n)$ nhưng vẫn còn hạn chế như: chưa khắc phục được vấn đề của First Come First Serve. Đó là khi một yêu cầu mới xuất hiện, nếu yêu cầu đó ở gần taxi A nhưng taxi A đã phục vụ một yêu cầu khác và di chuyển đi mất, taxi A đó không thể nào phục vụ được yêu cầu mới. Vì vậy, một taxi B khác cần được điều động tới phục vụ yêu cầu mới này, gây ra tình trạng có thể thiếu hụt xe để phục vụ tất cả yêu cầu.

Từ đó, khoá luận này cải tiến Linear Plain và đề xuất thuật toán Prune Prophet vẫn giữ được độ phức tạp thời gian là $O(n)$ và tối ưu hơn về số lượng taxi cần sử dụng bằng cách tận dụng dữ liệu dự đoán về yêu cầu đặt xe.

Chương 3. THIẾT KẾ HỆ THỐNG VÀ THUẬT TOÁN LẬP LỊCH

3.1 Tổng quan về hệ thống

Nhằm khắc phục những hạn chế của các phương pháp hiện có, chúng tôi đề xuất một hệ thống hỗ trợ dịch vụ đi chung xe trực tuyến. Hệ thống này hoạt động dựa trên việc phân tích dữ liệu không gian và thời gian về nhu cầu đặt xe, từ đó dự đoán và tạo ra các nhóm đi chung tiềm năng. Khi hành khách yêu cầu dịch vụ đi chung, hệ thống sẽ tìm kiếm và ghép họ vào nhóm phù hợp nhất đã được dự đoán trước đó. Cơ chế này đảm bảo mỗi nhóm sẽ có nhiều người tham gia, đồng thời tối ưu hóa lộ trình di chuyển của xe. Nhờ vậy, chi phí cho mỗi hành khách sẽ giảm đáng kể, và khả năng ghép nhóm thành công cũng được nâng cao rõ rệt. Phương pháp này hứa hẹn mang lại hiệu quả cao hơn so với các giải pháp trước đây. Khóa luận và hệ thống này tập trung vào việc xử lý bài toán thực tế của các hệ thống đặt xe nói chung và bài toán đi chung xe nói riêng, khi mà cần đưa trải nghiệm người dùng lên cao nhất thì phải giảm thời gian chờ đợi dịch vụ phản hồi. Vì lẽ đó, tôi đưa ra một hệ thống có thể xử lý yêu cầu thời gian thực của người dùng một cách nhanh nhất mà vẫn đảm bảo chi phí là tốt nhất.

3.1.1 Giới thiệu về hệ thống

Hệ thống do tôi xây dựng bao gồm ba tác nhân chính là: quản trị viên, tài xế và hành khách. Bảng dưới mô tả tác nhân và chức năng của mỗi tác nhân khi tham gia vào hệ thống.

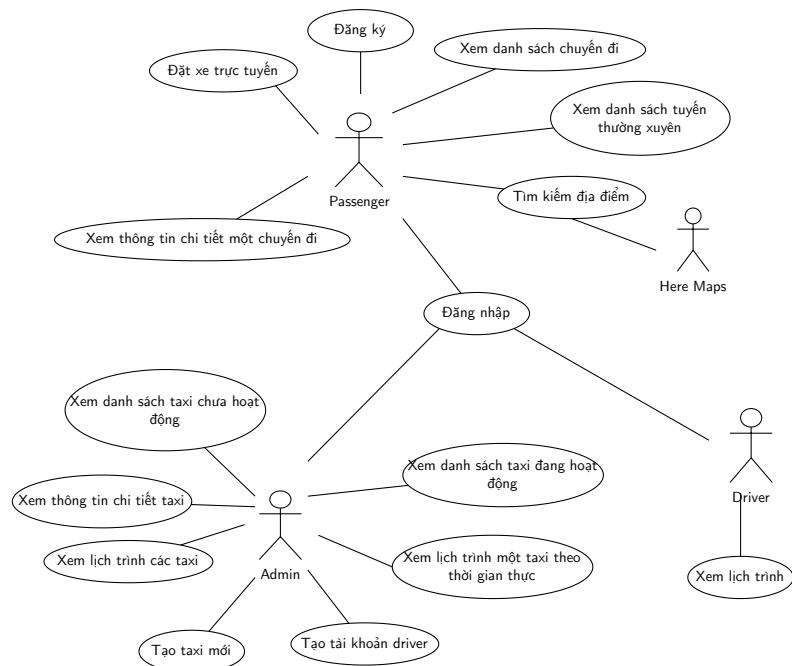
Bảng 3.1: Danh sách tác nhân tham gia hệ thống

Tác nhân	Mô tả
Tài xế	Tài xế của công ty taxi, mỗi tài xế sử dụng một taxi của công ty. Sử dụng để xem và đi theo lịch trình, phục vụ yêu cầu khách hàng theo hệ thống chỉ định
Hành khách	Sử dụng hệ thống với mục đích chính: + Yêu cầu đi chung xe thời gian thực + Yêu cầu đi xe một mình thời gian thực
Quản trị viên	Sử dụng hệ thống với ba mục đích chính: + Quản lý danh sách taxi của công ty + Quản lý tài khoản tài xế của công ty + Nhập dữ liệu tiên đoán nhu cầu đi chung xe vào hệ thống + Xem thông tin lịch trình của các xe taxi + Xem báo cáo về dữ liệu các chuyến đi, taxi, hành khách

3.1.2 Xác định các yêu cầu của hệ thống

Dưới đây là biểu đồ usecase tổng quát và đặc tả một số chức năng chính của hệ thống. Phụ lục A chứa danh sách đặc tả tất cả chức năng có trong hệ thống.

3.1.2.1 Biểu đồ usecase



Hình 3.1: Biểu đồ use case tổng quát của hệ thống

3.1.2.2 Đặc tả use case

Bảng 3.2: Chức năng xem lịch trình hiện tại

Id: 1	Tên: Xem lịch trình hiện tại	
Mô tả	Tài xế xem lịch trình hiện tại mình cần thực hiện	
Actor	Tài xế	
Luồng sự kiện thông thường	Actor 1. Ở giao diện trang chủ app di động sau khi đăng nhập, người dùng chọn phần trang chủ	System 2. Hệ thống hiển thị lịch trình hiện tại của tài xế với bản đồ thời gian thực, hiển thị đầy đủ điểm đi tiếp theo, hướng di chuyển, ...
Luồng sự kiện phụ	2.a Nếu taxi chưa có lịch trình nào thì hiển thị thông báo chưa có lịch trình nào	
Tiền điều kiện	Người dùng đã đăng nhập vào app hệ thống với vai trò tài xế	
Hậu điều kiện	Không có	

Bảng 3.3: Chức năng xem lịch trình các taxi theo thời gian thực

Id: 2	Tên: Xem lịch trình các taxi hiện tại	
Mô tả	Quản trị viên xem lịch trình các taxi đang hoạt động	
Actor	Quản trị viên	
Luồng sự kiện thông thường	Actor 1. Ở giao diện trang chủ web sau khi đăng nhập, người dùng chọn phần trang chủ 3. Người dùng chọn một taxi đang hoạt động để xem lịch trình của taxi đó	System 2. Hệ thống hiển thị danh sách các taxi đang hoạt động trên giao diện 4. Hệ thống hiển thị lịch trình, vị trí của taxi đã chọn trên bản đồ
Luồng sự kiện phụ	Không có	
Tiền điều kiện	Người dùng đã đăng nhập vào app hệ thống với vai trò quản trị viên	
Hậu điều kiện	Không có	

Bảng 3.4: Chức năng xem thống kê của hệ thống

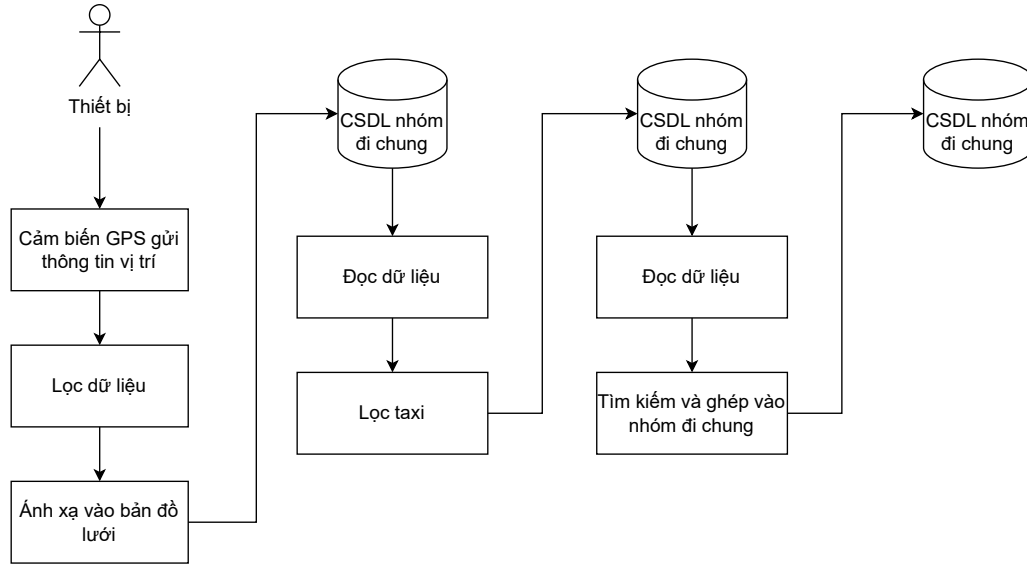
Id: 3	Tên: Xem thống kê các yêu cầu đã phục vụ, số lượng taxi đang sử dụng	
Mô tả	Quản trị viên xem thống kê, phân tích số liệu hệ thống	
Actor	Quản trị viên	
Luồng sự kiện thông thường	Actor 1. Ở giao diện trang chủ web sau khi đăng nhập, người dùng chọn phần trang chủ	System 2. Hệ thống hiển thị thống kê, các số liệu dịch vụ trên giao diện
Luồng sự kiện phụ	Không có	
Tiền điều kiện	Người dùng đã đăng nhập vào app hệ thống với vai trò quản trị viên	
Hậu điều kiện	Không có	

Bảng 3.5: Chức năng tạo tài khoản tài xế

Id: 3	Tên: Tạo tài xế mới cho hệ thống	
Mô tả	Quản trị viên dựa trên thông tin tài xế cung cấp tạo tài khoản cho tài xế	
Actor	Quản trị viên	
Luồng sự kiện thông thường	Actor 1. Ở giao diện trang chủ web sau khi đăng nhập, người dùng chọn phần tạo tài xế 3. Người dùng điền các thông tin của tài xế và chọn loại xe 5. Người dùng bấm gửi thông tin cho hệ thống	System 2. Hệ thống hiển thị biểu mẫu nhập thông tin tài xế 4. Hệ thống xác minh các trường dữ liệu đúng chuẩn 6. Hệ thống xác nhận đã tạo tài khoản tài xế thành công
Luồng sự kiện phụ	3a. Hệ thống xác minh dữ liệu sai chuẩn 6a. Hệ thống không thể tạo được tài khoản tài xế	
Tiền điều kiện	Người dùng đã đăng nhập vào app hệ thống với vai trò quản trị viên	
Hậu điều kiện	Không có	

3.2 Mô hình hệ thống

Dưới đây là mô hình của hệ thống cho chức năng ghép nhóm người dùng đặt xe trực tuyến.



Hình 3.2: Mô hình hệ thống cho chức năng tìm kiếm chuyến đi chung trực tuyến

Đầu vào của hệ thống là dữ liệu được thu thập từ cảm biến GPS từ những chiếc điện thoại di động của người dùng. Những dữ liệu này được thu thập nếu người dùng đồng ý chia sẻ vị trí. Khi người dùng thực hiện yêu cầu đặt xe có điểm đón là vị trí hiện tại, cảm biến sẽ lấy dữ liệu vị trí hiện tại của người dùng và gửi lên server cùng với các dữ liệu khác về yêu cầu đi chung xe. Khi server nhận được dữ liệu, hệ thống sẽ thực hiện ánh xạ thông tin về vị trí đó lên bản đồ lưới và thực hiện gán nhãn.

Khi quá trình dán nhãn dữ liệu hoàn thành, tiếp theo là việc tìm kiếm chuyến đi phù hợp cho yêu cầu đi chung xe. Hệ thống sẽ thực hiện tìm ra taxi phù hợp nhất để chèn một cặp điểm đi, điểm đến của yêu cầu đặt xe vào lịch trình của các taxi và tìm ra vị trí chèn cũng như taxi phù hợp nhất dựa trên tiêu chí quãng đường tăng lên nhỏ nhất. Ở đây tôi sử dụng bốn thuật toán có cách tiếp cận khác nhau và độ phức tạp từ $O(n^3)$ tới $O(n)$ để so sánh hiệu năng về thời gian là Basic Plain, Linear Plain, Prophet[9] và Prune Prophet.

Với thuật toán Prophet, trước đó hệ thống sẽ tiền xử lý dữ liệu yêu cầu được tiên đoán trước lấy từ nguồn/mô hình dự đoán dữ liệu, sau đó lập lịch cho các taxi đi tới các nơi có thể xuất hiện yêu cầu dự đoán.

Cuối cùng, hệ thống sẽ chèn cặp điểm đi, điểm đến của yêu cầu vào vị trí tối ưu nhất

của lịch trình taxi tốt nhất (tùy vào vị trí hiện tại của taxi) và lưu vào cơ sở dữ liệu. Đồng thời thông báo cho cả người dùng và tài xế taxi đó.

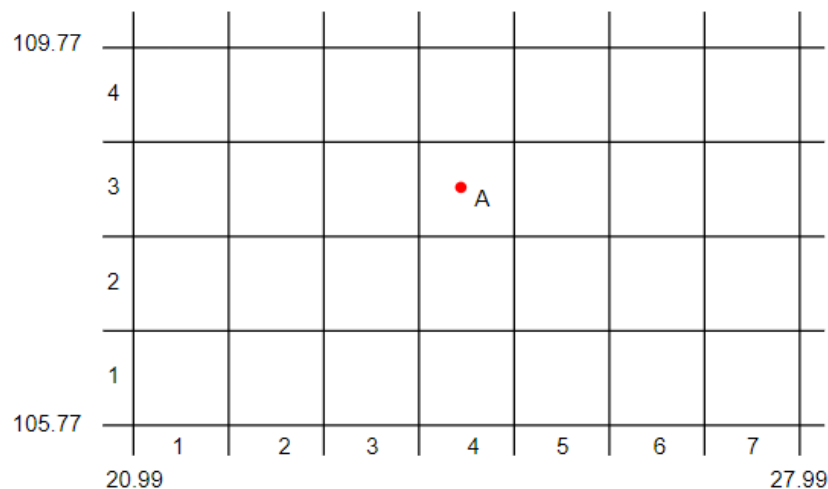
3.3 Xây dựng ô lưới không gian để biểu diễn vị trí

Trong bài toán tối ưu hóa lộ trình đi chung, việc liên tục tính toán quãng đường ngắn nhất giữa các điểm để tiết kiệm chi phí tiêu tốn rất nhiều tài nguyên hệ thống. Để giải quyết vấn đề này, thay vì tính toán trực tiếp, chúng ta có thể sử dụng một phương pháp tính toán gần đúng dựa trên khoảng cách tương đối được xác định trước, như đề xuất trong [4]. Mặc dù không thể hoàn toàn chính xác, nhưng phương pháp này giúp giảm đáng kể thời gian xử lý, cải thiện trải nghiệm người dùng và giảm tải cho hệ thống. Cụ thể, bản đồ được chia thành một lưới các ô vuông, mỗi ô chứa một tập hợp các điểm giới hạn bởi tọa độ kinh độ và vĩ độ. Mỗi ô sẽ có một điểm neo đại diện, được lựa chọn dựa trên vị trí địa lý và cơ sở hạ tầng giao thông. Bất kỳ điểm nào trên bản đồ đều được ánh xạ vào một ô cụ thể trong lưới. Khi đó, việc tính toán quãng đường ngắn nhất giữa hai điểm sẽ được thay thế bằng việc tra cứu khoảng cách giữa hai ô lưới tương ứng, dựa trên khoảng cách giữa các điểm neo đã được tính toán trước. Nhờ vậy, độ phức tạp tính toán giảm xuống chỉ còn $O(1)$, giúp tối ưu hóa hiệu suất hệ thống.

3.4 Xây dựng lưới và xử lý dữ liệu điểm di chuyển

Với một yêu cầu đặt xe từ khách hàng bao gồm điểm đón và điểm trả, chúng ta thực hiện ánh xạ các điểm đó lên bản đồ lưới. Dữ liệu sẽ được ánh xạ dưới dạng:

$$P\{x, y\}, \text{ với } \begin{cases} x : \text{kinh độ} \\ y : \text{vĩ độ} \end{cases}$$



Hình 3.3: Ví dụ minh họa ánh xạ một điểm lên bản đồ lưới

Hình vẽ minh họa cách thức ánh xạ một điểm lên lưới địa lý. Lưới này có kích thước 7x4, với trục hoành biểu diễn kinh độ (khởi đầu từ 20.99) và trục tung biểu diễn vĩ độ (khởi đầu từ 105.77). Mỗi ô vuông trên lưới có cạnh là 1°. Điểm A, với kinh độ 24.3 và vĩ độ 108.01, được ánh xạ tới ô P(4, 3) trên lưới. Khoảng cách ngắn nhất giữa các cặp điểm trong một khu vực cụ thể được tính toán trước và lưu trữ trong cơ sở dữ liệu quan hệ dưới dạng `des[p][q]`, trong đó `des` biểu thị khoảng cách ngắn nhất từ điểm `p` đến điểm `q`. Để đáp ứng nhu cầu truy xuất dữ liệu liên tục, đặc biệt trong giờ cao điểm, một máy chủ cache được triển khai phía trước cơ sở dữ liệu, chứa dữ liệu cho một khu vực nhỏ nhất định. Giải pháp này giúp tối ưu hóa đáng kể việc tính toán khoảng cách trong các thuật toán, đảm bảo hiệu suất hệ thống trong điều kiện thực tế.

3.5 Cơ sở dữ liệu

Cơ sở dữ liệu của tôi được phát triển dựa trên nền tảng cơ sở dữ liệu trong khóa luận [4] và bài báo [15]. Ở cơ sở dữ liệu cũ lưu trữ các dữ liệu liên quan đến thông tin tuyến đường, lịch sử di chuyển, dữ liệu tuyến đường đã ánh xạ lên bản đồ lưới, tuyến đường thường xuyên với mục đích là lưu lại lịch sử di chuyển của người dùng. Dựa trên nền tảng cơ sở dữ liệu đó, cơ sở dữ liệu mới của tôi được thiết kế thêm để tính toán thời gian chờ (waiting time) của lịch sử di chuyển đưa vào làm bộ dữ liệu dự đoán trước, có thể lưu lại thông tin yêu cầu đi chung xe trực tuyến và các dữ liệu cần thiết nhằm thực hiện các thuật toán chèn một cặp điểm đi, điểm đón của yêu cầu đi chung xe trực tuyến.

Các thực thể chính trong lược đồ cơ sở dữ liệu quan hệ của hệ thống:

- **request_ride**: Chứa những yêu cầu mới từ người dùng.
- **predicted_request**: Chứa những yêu cầu được dự đoán của hệ thống khác.
- **group_frequent**: Chứa thông tin về những nhóm/xe đi chung.
- **taxi**: Thông tin về xe, số ghế ngồi.
- **guidance_schedule**: Lịch trình của nhóm đi chung.
- **grid**: Lưới dữ liệu bản đồ.
- **edge**: Thông tin về khoảng cách ngắn nhất giữa 2 ô trong lưới bản đồ.
- **admin, passenger, driver**: Quản lý thông tin tài khoản.

Phụ lục [A](#) chứa chi tiết các bảng trong cơ sở dữ liệu.

cải tiến để chen với dữ liệu tiên đoán trước: Prophet, Prune Prophet. Riêng nhóm thuật toán Prophet sẽ cần thêm dữ liệu dự đoán yêu cầu đi xe của người dùng. Thuật toán chen với dữ liệu tiên đoán trước (Prophet) là cải tiến của thuật toán Plain để có thể tận dụng được phần dữ liệu tiên đoán này làm cho việc điều phối taxi thông minh hơn. Để bài toán đơn giản hơn, tôi coi tốc độ di chuyển của taxi là không đổi trong suốt lịch trình. Do đó, khoảng cách di chuyển tăng thêm cũng là thời gian di chuyển tăng thêm.

3.6.1 Các khái niệm

Mạng lưới giao thông: biểu diễn bằng một đồ thị vô hướng $\mathbf{G} = (\mathbf{V}, \mathbf{E})$ với tập đỉnh V và tập cạnh E . Mỗi cạnh $(\mathbf{u}, \mathbf{v}) \in E$ bao gồm độ dài đường đi từ u đến v và thời gian để taxi đi từ u đến v trong điều kiện giao thông bình thường.

Taxi: mỗi taxi được biểu diễn là $\mathbf{w} = \langle \mathbf{o}_w, \mathbf{c}_w \rangle$ với $\mathbf{o}_w$ là vị trí của taxi và c_w là sức chứa của taxi đó.

Yêu cầu đặt xe: mỗi yêu cầu được biểu diễn là $\mathbf{r} = \langle \mathbf{o}_r, \mathbf{d}_r, \mathbf{c}_r, \mathbf{l}_r, \mathbf{t}_r \rangle$ với $\mathbf{o}_r$ là vị trí điểm đón, $\mathbf{d}_r$ là vị trí điểm trả, $\mathbf{c}_r$ là số lượng chỗ ngồi cần phục vụ, $\mathbf{l}_r$ là thời gian muộn nhất taxi cần đến điểm đón $\mathbf{o}_r$ và $\mathbf{t}_r$ là thời gian yêu cầu xuất hiện. Ngoài ra chúng ta sử dụng $\mathbf{r}_w = \{\mathbf{r}_1, \mathbf{r}_2, \dots, \mathbf{r}_n\}$ để biểu diễn tập các yêu cầu đặt xe được phục vụ bởi taxi w .

Lịch trình taxi: lịch trình của một taxi w được biểu diễn là $\mathbf{s}_w = \langle \mathbf{o}_w, \mathbf{l}_w^1, \dots, \mathbf{l}_w^{|\mathbf{s}_w|-1} \rangle$, trong đó $\langle \mathbf{l}_w^1, \dots, \mathbf{l}_w^{|\mathbf{s}_w|-1} \rangle$ là một dãy có thứ tự $\mathbf{o}_r$ và $\mathbf{d}_r$ của $\mathbf{r}_w$, nghĩa là $\mathbf{l}_w^i \in \{\mathbf{o}_r | \mathbf{r} \in \mathbf{r}_w\} \cup \{\mathbf{d}_r | \mathbf{r} \in \mathbf{r}_w\}$.

1 Một lịch trình là **khả thi** nếu:

- i) $\forall \mathbf{r} \in \mathbf{r}_w, \mathbf{o}_r$ và $\mathbf{d}_r$ tồn tại và $\mathbf{o}_r$ đứng trước $\mathbf{d}_r$ trong dãy
- ii) Tại bất kỳ thời điểm nào, số lượng hành khách trên xe cũng **không được vượt quá** dung lượng của taxi

Các định nghĩa mới của thuật toán Prophet:

- **Yêu cầu được dự đoán trước:** là một yêu cầu đặt xe được lấy từ dữ liệu được dự đoán trước được đề cập ở phần 3.2, thêm trường **wait_r** hay w_r là thời gian chờ tính từ lúc taxi đến điểm đón cho tới khi yêu cầu thực sự xuất hiện.

Tập $\tilde{R}$ là tập những yêu cầu đã dự đoán (sau đây gọi là yêu cầu dự đoán).

- **Lịch trình hướng dẫn taxi đi tới điểm có yêu cầu dự đoán:** lịch trình hướng dẫn của taxi $\tilde{\mathbf{w}}$ được biểu diễn là

$$\tilde{\mathbf{s}}_{\mathbf{w}} = \langle \langle \mathbf{o}_{\mathbf{w}}, \mathbf{w}_{\mathbf{r}} \rangle, \langle \mathbf{l}_{\mathbf{w}}^1, \mathbf{w}_1 \rangle, \dots, \langle \mathbf{l}_{\mathbf{w}}^{|\mathbf{s}_{\mathbf{w}}|-1}, \mathbf{w}_{|\mathbf{s}_{\mathbf{w}}|-1} \rangle \rangle,$$

trong đó $\langle \langle l_w^1, w_1 \rangle, \dots, \langle l_w^{|\mathbf{s}_w|-1}, w_{|\mathbf{s}_w|-1} \rangle \rangle$ là một dãy có thứ tự $\mathbf{o}_{\mathbf{r}}$ và $\mathbf{d}_{\mathbf{r}}$ của $\mathbf{r}_{\mathbf{w}}$, nghĩa là $\mathbf{l}_{\mathbf{w}}^i \in \{o_r | r \in \tilde{r}_w\} \cup \{d_r | r \in \tilde{r}_w\}$, $\mathbf{w}_i$ là thời gian đợi của taxi khi ở yên tại vị trí l_i .

Một lịch trình hướng dẫn là **khả thi** nếu:

1. $\forall r \in \tilde{r}_w$, o_r và d_r tồn tại và o_r đứng trước d_r
2. Tại bất kỳ thời điểm nào, số lượng hành khách trên xe cũng không được vượt quá dung lượng của taxi
3. $\forall r \in \tilde{r}_w$, thời gian taxi w rời khỏi o_r không sớm hơn thời gian yêu cầu xuất hiện t_r và thời gian taxi w đến điểm trả d_r không muộn hơn thời gian deadline của yêu cầu.

3.6.2 Thuật toán chèn cơ bản

a) Ý tưởng thuật toán

Thuật toán dựa trên việc chèn yêu cầu (Plain) được đề cập trong các nghiên cứu trước đây dựa trên một ý tưởng đơn giản của việc chèn cặp điểm đón, trả vào lịch trình có sẵn:

- i) Liệt kê tất cả các điểm (o_r, d_r) có thể chèn vào lịch trình của một taxi để tạo ra lịch trình mới S'_w
- ii) Kiểm tra xem lịch trình mới S'_w có vi phạm bất kỳ ràng buộc nào hay không
- iii) Lưu lại lịch trình mới S'_w và tìm ra S'_w có thời gian di chuyển tăng lên ngắn nhất

b) Chi tiết thuật toán

Thuật toán 1: Thuật toán chèn cơ bản - Basic Plain

Input: a worker w with route S_w and a request r

Output: a new route S^* for the worker w

```
1  $S^* \leftarrow S, \Delta^* \leftarrow \infty$ 
2 for  $i \leftarrow 0$  to  $n$  do
3   for  $j \leftarrow i$  to  $n$  do
4      $S' \leftarrow$  insert  $o_r$  at  $i$  and  $d_r$  at  $j$  in  $S$ 
5     if  $S'$  is feasible then
6        $\Delta_{i,j} \leftarrow D(S') - D(S)$ 
7       if  $\Delta_{i,j} < \Delta^*$  then
8          $S^* \leftarrow S'$ 
9          $\Delta^* \leftarrow \Delta_{i,j}$ 
10 return  $S^*$ 
```

Ở dòng 1 khởi tạo S^* để lưu lại lịch trình tối ưu nhất, trong trường hợp không có lịch trình mới nào khả thi thì S^* có giá trị là S ban đầu của taxi, và khởi tạo Δ^* để lưu lại thời gian di chuyển tăng lên tối thiểu. Các dòng tiếp theo là 2 và 3, tôi sử dụng 2 vòng lặp để duyệt qua tất cả các điểm có thể chèn (với i là vị trí chèn o_r và j là vị trí chèn d_r). Từ dòng 4 đến 7 sẽ được thực hiện ở bên trong hai vòng lặp. Đầu tiên ở dòng 4, tạo một biến cục bộ S' để lưu lịch trình mới khi chèn o_r vào vị trí i và d_r vào vị trí j của lịch trình ban đầu S . Tiếp theo ở dòng 5, tôi kiểm tra xem S' có khả thi hay không.

Thuật toán 2: Kiểm tra độ khả thi của lịch trình

Input: a worker w with route S

Output: is S a feasible schedule

```
1 for  $i \leftarrow 0$  to  $S.length - 1$  do
2   if capacity of  $w$  at  $i < 0$  then
3     return false
4   if time expected to arrive  $i < late\ time\ to\ i$  then
5     return false
6 return true
```

Mã giả trên kiểm tra xem lịch trình S của taxi w có khả thi hay không. Nếu khả thi thì trả về true, ngược lại trả về false. Ở đây, tôi sử dụng duy nhất một vòng lặp để duyệt qua tất cả các điểm có trong S . Với mỗi điểm duyệt qua, tôi thực hiện kiểm tra số lượng chỗ ngồi còn lại của taxi ở điểm i có nhỏ hơn 0 hay không và thời gian taxi đến điểm i

có trễ hơn thời gian muộn nhất taxi cần đến điểm i hay không. Nếu S' khả thi, tiếp theo thực hiện tính toán khoảng cách tăng thêm của S' so với S ban đầu.

Thuật toán 3: Tính tổng thời gian

Input: a worker w with route S

Output: total time of S

```

1  $D(S) \leftarrow 0$ 
2 for  $i \leftarrow 0$  to  $S.length - 2$  do
3    $D(S) \leftarrow D(S) + time(S_i, S_{i+1})$ 
4 return  $D(S)$ 
```

Mã giả trên tính tổng thời gian di chuyển của một lịch trình bất kỳ. Dòng 7, so sánh $\Delta_{i,j}$ vừa tính được với Δ^* , nếu nhỏ hơn có nghĩa là lịch trình mới tìm được tối ưu hơn. Nên tôi thực hiện gán S^* bằng S' và Δ^* bằng $\Delta_{i,j}$.

c) Phân tích độ phức tạp thuật toán

Tôi sử dụng hai vòng lặp để duyệt tất cả cặp (i, j) nên số cặp có thể có là $O(n^2)$. Phần kiểm tra xem lịch trình mới có khả thi hay không, sử dụng một vòng lặp để duyệt qua tất cả các điểm trong lịch trình nên độ phức tạp là $O(n)$. Nếu lịch trình khả thi sẽ tính thời gian tăng lên, để tính được tổng thời gian di chuyển của mỗi lịch trình cần một vòng lặp duyệt qua tất cả các điểm của lịch trình nên độ phức tạp là $O(n)$. Về phần tính khoảng cách ngắn nhất và thời gian ngắn nhất đi từ điểm A bất kỳ đến điểm B, ở phần trên tôi đã đề cập đến việc sử dụng bản đồ lưới và các điểm neo trong nghiên cứu này nên chỉ tốn thời gian $O(1)$. Do đó độ phức tạp về thời gian của thuật toán Basic Insertion là $O(n^3)$ và chi phí cần cho bộ nhớ là $O(n)$.

3.6.3 Thuật toán chèn tuyến tính - Linear Plain

a) Ý tưởng thuật toán

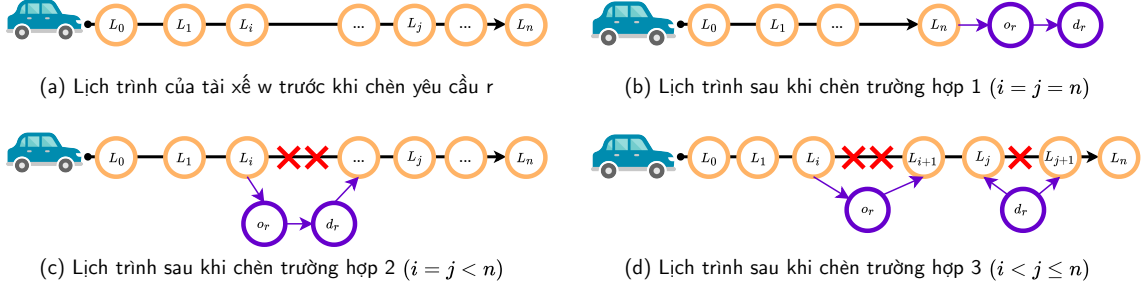
i) Cải tiến từ thuật toán Basic Plain, không còn liệt kê tất cả các cặp vị trí (i, j) có thể chèn nữa. Mà ý tưởng ở đây dựa trên hai ý chính:

- Với trường hợp $i = j$ chỉ tốn thời gian $O(n)$ để tìm ra tuyến đường tốt nhất.
- Ở các trường hợp còn lại: với điểm j cố định (j là vị trí chèn d_r) chỉ mất thời gian $O(1)$ để tìm ra vị trí i tốt nhất (i là vị trí chèn o_r).

ii) Kiểm tra lịch trình mới S'_w có khả thi hay không và tính toán thời gian di chuyển tăng thêm của S'_w so với S_w là $\Delta_{i,j}$ trong thời gian $O(1)$ thay vì thời gian $O(n)$ như trong thuật toán Basic Plain.

b) Chi tiết thuật toán

Kiểm tra tính khả thi của lịch trình mới S'_w và tính toán thời gian tăng thêm $\Delta_{i,j}$ của S'_w so với S_w .



Hình 3.5: Các trường hợp khi chèn o_r và d_r

Có tất cả ba trường hợp có thể xảy ra khi thực hiện chèn o_r và d_r vào lịch trình S_w . Trong đó: Trường hợp 1 (Hình b): vị trí chèn o_r, d_r trùng nhau và nằm cuối cùng của lịch trình S_w . Trường hợp 2 (Hình c): vị trí chèn o_r, d_r trùng nhau và nằm giữa hai điểm bất kỳ trong lịch trình S_w . Trường hợp 3 (Hình d): là trường hợp tổng quát khi vị trí chèn o_r, d_r khác nhau và mỗi điểm nằm ở giữa hai điểm bất kỳ trong lịch trình S_w .

Tính $\Delta_{i,j}$ trong thời gian $O(1)$

Ở đây tôi sử dụng khái niệm detour để tính thời gian di chuyển tăng lên khi chèn điểm l_j vào giữa l_i và l_k . Công thức cụ thể như sau: $det(l_i, l_j, l_k) = dis(l_i, l_j) + dis(l_j, l_k) - dis(l_i, l_k)$ Trong đó để tính được $dis(A, B)$ là thời gian di chuyển ít nhất để đi từ A đến B chỉ tốn thời gian $O(1)$ vì tôi sử dụng bản đồ lưới và các điểm neo. Kết hợp detour và ba trường hợp khi chèn o_r, d_r ta có công thức tính $\Delta_{i,j}$ trong thời gian $O(1)$ như sau:

$$\Delta_{i,j} = \begin{cases} dis(l_n, o_r) + dis(o_r, d_r) & i = j = n \\ dis(l_i, o_r) + dis(o_r, d_r) + dis(d_r, l_{i+1}) - dis(l_i, l_{i+1}) & i = j < n \\ det(l_i, o_r, l_{i+1}) + det(l_j, d_r, l_{j+1}) & otherwise \end{cases}$$

Kiểm tra tính khả thi của S_w trong thời gian $O(1)$

Lịch trình S_w khả thi là một lịch trình đáp ứng đủ hai điều kiện: giới hạn về thời gian và dung lượng. Ý tưởng ở đây là sử dụng khái niệm slack time để biểu diễn thời gian dư thừa tại mỗi điểm. Ví dụ: xét một điểm i trong lịch trình S_w , thời gian muộn nhất taxi cần đến i là 10:30 và thời gian dự kiến taxi đến i là 10:15. Từ đây tính được slack time

tại điểm i là $10:30 - 10:15 = 15$ phút. Một số khái niệm cần để tính slack time tại một điểm:

Thời gian muộn nhất taxi cần đến điểm l_k

$$ddl[k] = \begin{cases} e_r - dis(o_r, d_r) & l_k \text{ is } o_r \\ e_r & l_k \text{ is } d_r \end{cases}$$

Thời gian taxi dự kiến đến điểm l_k

$$arr[k] = arr[k-1] + dis(l_{k-1}, l_k)$$

Ngoài ra, biểu diễn $slack[k]$ là thời gian trễ tối đa giữa l_k và l_{k+1} để không làm trễ tất cả các điểm phía sau l_k . Từ đây ta có công thức tính slack time tại điểm l_k

$$\begin{aligned} slack[k] &= \min_{k' \rightarrow k} (ddl[k'] - arr[k']) \\ &= \min\{slack[k+1], ddl[k+1] - arr[k+1]\} \end{aligned}$$

Từ đây ta có điều kiện ràng buộc về thời gian. Để lịch trình S_w khả thi cần thỏa mãn bốn điều kiện sau:

- i) $arr[i] + dis(l_i, o_r) \leq e_r$
- ii) $det(l_i, o_r, l_{i+1}) \leq slack[i]$
- iii) $\begin{cases} arr[i] + dis(l_i, o_r) + dis(o_r, d_r) \leq e_r & i = j \\ arr[j] + det(l_i, o_r, l_{i+1}) + dis(l_j, d_r) \leq e_r & i < j \end{cases}$
- iv) $\Delta_{i,j} \leq slack[j]$

Ở điều kiện (i) kiểm tra xem taxi có đến o_r trước thời hạn muộn nhất của yêu cầu là e_r hay không nếu o_r được chèn tại vị trí thứ i . Điều kiện (ii) kiểm tra xem khi chèn o_r vào vị trí thứ i thì thời gian tăng thêm đó có làm các điểm phía sau i vi phạm về thời gian hay không. Điều kiện (iii) tương tự điều kiện (i) chỉ khác là kiểm tra ràng buộc khi chèn d_r . Điều kiện (iv) kiểm tra xem các điểm phía sau vị trí j (vị trí chèn d_r) có vi phạm ràng buộc về thời gian hay không.

Sau khi kiểm tra ràng buộc về thời gian. Tiếp theo tôi sẽ thực hiện kiểm tra ràng buộc về dung lượng trong thời gian $O(1)$. Ý tưởng ở đây là sử dụng mảng một chiều picked để lưu trữ số lượng chỗ còn trống của taxi tại mỗi điểm:

$$picked[j] = \begin{cases} picked[k-1] + k_r & l_k \text{ is } o_r \\ picked[k-1] & l_k \text{ is } d_r \end{cases}$$

Từ đây ta có điều kiện ràng buộc về dung lượng. Để lịch trình S_w khả thi cần thỏa mãn hai điều kiện sau:

$$\text{i) } picked[i] \leq k_w - k_r$$

$$\text{ii) } \forall k, i < k \leq j, picked[k] \leq k_w - k_r$$

Ở điều kiện (i) kiểm tra xem tại vị trí chèn o_r là điểm i thì taxi còn đủ chỗ trống để phục vụ yêu cầu hay không. Điều kiện (ii) kiểm tra tất cả các điểm giữa i và j (bao gồm cả điểm j là vị trí chèn d_r) có thỏa mãn ràng buộc về dung lượng hay không.

Liệt kê vị trí d_r

Thay vì liệt kê tất cả các cặp (i, j) như thuật toán trước, ở thuật toán này chỉ liệt kê các vị trí j để tìm ra tuyến đường tốt nhất. Để làm được điều này, tôi sử dụng Δ_j biểu diễn thời gian di chuyển tăng lên tối thiểu với một j cố định:

$$\begin{aligned} \Delta_j &= \min \Delta_{i,j} \\ &= \min(\det(l_i, o_r, l_{i+1}) + \det(l_j, d_r, l_{j+1})) \\ &= \det(l_j, d_r, l_{j+1}) + \min(\det(l_i, o_r, l_{i+1})) \end{aligned}$$

Nhìn vào công thức trên, có thể thấy $\det(l_j, d_r, l_{j+1})$ là hằng số cố định vì j cố định. Nên cốt lõi của thuật toán Linear Plain là tìm vị trí i khả thi để cực tiểu hóa $\det(l_i, o_r, l_{i+1})$ trong thời gian $O(1)$.

Tìm vị trí chèn o_r tốt nhất trong thời gian $O(1)$

Để thực hiện được việc này, có một số khái niệm cần dùng đến. Đầu tiên là với một j cố định thì thời gian tăng lên tối thiểu khi chèn o_r ở các vị trí $i < j$.

$$Dio[j] = \begin{cases} \infty & picked[j-1] > k_w - k_r \\ Dio[j-1] & \det(l_{j-1}, o_r, l_j) > slack[j-1] \\ \min\{Dio[j-1], \det(l_{j-1}, o_r, l_j)\} & otherwise \end{cases}$$

Tiếp theo tôi sử dụng Plc để ghi lại vị trí i (vị trí chèn o_r) tương ứng với $Dio[j]$

$$Plc[j] = \begin{cases} NIL & picked[j-1] > k_w - k_r \\ Plc[j-1] & \det(l_{j-1}, o_r, l_j) > slack[j-1] \\ Plc[j-1] & Dio[j-1] < \det(l_{j-1}, o_r, l_j) \\ j-1 & Dio[j-1] \geq \det(l_{j-1}, o_r, l_j) \end{cases}$$

c) Phân tích độ phức tạp thuật toán

Thuật toán 4: Thuật toán chèn tuyến tính - Linear Plain

Input: a worker w with route S_w and a request r

Output: a new route S^* for the worker w

```

1  $S^* \leftarrow S, \Delta^* \leftarrow \infty, Dio[0] \leftarrow \infty, Plc[0] \leftarrow NIL$ 
2 Initialize  $ddl, arr, slack, picked$ 
3 for  $j \leftarrow 0$  to  $n$  do
4    $\Delta_j \leftarrow dis(l_j, o_r) + dis(o_r, d_r) + dis(d_r, l_{j+1}) - dis(l_j, l_{j+1})$ 
5   if  $\Delta_j < \Delta^*$  then
6      $\Delta^* \leftarrow \Delta_j$ 
7      $i^* \leftarrow j$ 
8      $j^* \leftarrow j$ 
9   if  $j > 0$  and  $picked[j] \leq k_w - k_r$  and  $arr[j] + Dio[j] + dis(l_j, d_r) \leq e_r$  and
     $Dio[j] + det(l_j, d_r, l_{j+1}) \leq slack[j]$  then
10     $\Delta_j \leftarrow Dio[j] + det(l_j, d_r, l_{j+1})$ 
11    if  $\Delta_j < \Delta^*$  then
12       $\Delta^* \leftarrow \Delta_j$ 
13       $i^* \leftarrow Plc[j]$ 
14       $j^* \leftarrow j$ 
15    if  $arr[j] + dis(l_j, d_r) > e_r$  then
16      break
17    Update  $Dio[j + 1]$  and  $Plc[j + 1]$  according above recipe
18 if  $\Delta^* < \infty$  then
19    $S^* \leftarrow \text{insert } o_r \text{ at } i^* \text{ and } d_r \text{ at } j^* \text{ in } S$ 
20 return  $S^*$ 

```

Ở dòng đầu tiên, giống như thuật toán Basic Plain, Tôi khởi tạo S^* và Δ^* để lưu trữ lịch trình tối ưu nhất và thời gian tăng lên nhỏ nhất tương ứng. Ngoài ra gán giá trị thêm cho hai biến là phần tử đầu tiên của hai mảng Dio , Plc lần lượt là ∞ và NIL (không xác định). Vì ở phần tử đầu tiên có nghĩa là $j = 0$ thì không có trường hợp nào có thể chèn điểm $i < j$. Tiếp theo ở dòng 2, tôi khởi tạo các mảng ddl , arr , $slack$, $picked$. Dòng 3, thực hiện một vòng lặp để duyệt qua tất cả các điểm có thể để chèn d_r . Trong vòng lặp, dòng 5 thực hiện tính toán cho các điểm $i = j$, từ dòng 6 - 8 thực hiện tính toán để tìm ra vị trí chèn o_r tốt nhất với j cố định trong thời gian $O(1)$. Dòng 9 thực hiện kiểm tra nếu chèn d_r vào điểm i hiện tại thì có vượt quá thời gian muộn nhất cần đến điểm trả hay không, nếu vượt quá thì kết thúc vòng lặp và loại bỏ tất cả các điểm còn lại. Dòng cuối cùng trong vòng lặp (dòng thứ 10) thực hiện cập nhật giá trị cho phần tử tiếp theo

của mảng *Dio* và *Plc*. Sau khi kết thúc vòng lặp, từ dòng 11 - 12, tương tự như hai thuật toán trên. Kiểm tra xem có lịch trình tốt nhất hay không sau đó thực hiện chèn vào và trả về kết quả.

3.6.4 Thuật toán chèn với dữ liệu dự đoán - Prophet

a) Ý tưởng thuật toán

Dựa vào các định nghĩa mới về lịch trình dự đoán ở phần 3.6.1 Các khái niệm, ta có thể coi như thuật toán chèn thông thường (Plain) là trường hợp đặc biệt của thuật toán chèn với dữ liệu dự đoán (Prophet) khi tất cả các $\mathbf{w}_r$ đều bằng 0. Từ đó, ta có thể định nghĩa bài toán tối ưu lịch trình như sau:

Bài toán tiên đoán: Giả định một taxi $\mathbf{w}$ với lịch trình hướng dẫn $\mathbf{S}^*_{\mathbf{w}}$ chứa $n + 1$ điểm $(l_0, l_1, \dots, l_n)$ và một yêu cầu đi chung mới $\mathbf{r}$ gồm cặp điểm đón - trả: $o_r - d_r$, thuật toán Prophet tạo một lịch trình hướng dẫn $\mathbf{S}^*_{\mathbf{w}}$ khả thi với thời gian, khoảng cách đi thêm là nhỏ nhất bằng cách chèn cặp điểm o_r và d_r vào S_w , sao cho thứ tự của các vị trí trong S_w không thay đổi.

Từ đó bài toán được chia làm hai phần, phần thứ nhất xử lý những yêu cầu dự đoán trước, phần thứ hai xử lý những yêu cầu trực tuyến.

i) **Phần 1: Xử lý yêu cầu dự đoán:** Ý tưởng: với mỗi một yêu cầu dự đoán $\tilde{R}$, ta tạo lịch trình hướng dẫn $\tilde{S}_w$ cho mỗi một xe taxi hiện có như sau:

- Tạo ra các taxi ảo (worker $\tilde{W}$) với vị trí ban đầu là vị trí bắt đầu yêu cầu dự đoán
- Xử lý tất cả các yêu cầu dự đoán cho mỗi taxi đi chung, sao cho thời gian, quãng đường đi thêm là nhỏ nhất
- Chọn ra $|W|$ taxi ảo để lập lịch lịch trình hướng dẫn cho taxi thật W , với lịch trình hướng dẫn là lịch trình có số lượng yêu cầu được phục vụ là lớn nhất.

Thuật toán 5: Tiền xử lý các yêu cầu đã được dự đoán

Input: Workers W and a set of predicted requests $\tilde{R}$

Output: A guidance route $\tilde{S}_w$ for each worker $w \in W$

```
1  $\tilde{W} \leftarrow \emptyset$  ;
2 for next predicted request  $r \in \tilde{R}$  do
3    $Cand \leftarrow$  filter the infeasible workers in  $\tilde{W}$  by grid index;
4    $\tilde{w}^* \leftarrow \operatorname{argmin}_{\tilde{w} \in \tilde{W}} \text{prophet-insert}(\tilde{w}, r)$ ;
   if  $\tilde{w}^*$  is not empty then
5     Insert  $r$  into the guidance route of  $\tilde{w}^*$  by  $\text{prophet-insert}(\tilde{w}^*, r)$ ;
   else
6      $\tilde{w} \leftarrow$  a hypothetical worker at the location or to serve  $r$ ,  $\tilde{W} \leftarrow \tilde{W} \cup \{\tilde{w}\}$ ;
7  $Q \leftarrow$  a max-heap of  $\tilde{W}$  based on the total number/revenue of the served requests
   in the guidance route of  $\tilde{w} \in \tilde{W}$ ;
8 while  $Q$  is not empty and  $W$  is not empty do
9    $\tilde{w}^* \leftarrow$  pop the top element from  $Q$ ,  $\tilde{w} \leftarrow$  the worker in  $W$  who can serve all
   the requests in the guidance route of  $\tilde{w}^*$  with minimum increased travel time;
   if  $w$  is not empty then
10    Plan the guidance route of  $\tilde{w}$  based on  $w$ ,  $W \leftarrow W \setminus \{w\}$ ;
11    Remove the first request in the guidance route of  $\tilde{w}^*$ ,  $Q \leftarrow Q \cup \{\tilde{w}^*\}$ ;
```

Thuật toán 5 tiền xử lý các yêu cầu dự đoán trước. Đối với mỗi yêu cầu r trong $\tilde{R}$, đầu tiên chúng ta xác định một tập $Cand$ của các taxi $\tilde{w}$ ứng viên theo chỉ số lưới ở dòng 3. Sau đó, chúng ta áp dụng phương pháp chen dự đoán Prophet để tìm taxi $\tilde{w}^*$ trong $\tilde{W}$, taxi nào có thời gian di chuyển tăng ít nhất (Δ^*) để chen yêu cầu dự đoán r (dòng 4). Nếu tìm thấy taxi như vậy, chúng ta sẽ chen r vào lịch trình hướng dẫn của taxi đó ở dòng 5. Nếu không, chúng ta sẽ tạo ra một taxi tạm thời mới $\tilde{w}$ để phục vụ r ở dòng 6, với vị trí ban đầu là điểm xuất phát của r . Khi tất cả các yêu cầu dự đoán đã được chen, chúng ta sẽ gán các yêu cầu vào các lịch trình của các taxi tạm để xây dựng lịch trình hướng dẫn cho các taxi thật. Cụ thể, chúng ta sử dụng một heap max Q để duy trì taxi tạm $\tilde{w}^*$ với số lượng yêu cầu phục vụ lớn nhất trong các dòng 7–11. Đối với mỗi $\tilde{w}^*$, chúng ta thử lặp đi lặp lại mỗi taxi trong W để phục vụ tất cả các yêu cầu, rời khỏi điểm xuất phát vào thời gian 0 và sau đó đi theo lịch trình của $\tilde{w}^*$. Do đó, chúng ta có thể tìm được taxi w có thời gian di chuyển tăng nhỏ nhất để phục vụ các yêu cầu và lên kế hoạch lịch trình hướng dẫn cho taxi đó (dòng 10). Nếu taxi w không tồn tại, điều này có nghĩa là không có taxi nào có thể đến điểm xuất phát đầu tiên trong lịch trình $\tilde{w}^*$ đúng giờ.

Vì vậy, chúng ta sẽ loại bỏ yêu cầu đầu tiên đã được chọn trong lịch trình của $\tilde{w}^*$ và đưa taxi đó trở lại trong heap Q (dòng 11).

ii) **Phần 2: xử lý yêu cầu trực tuyến:**

Ý tưởng: với mỗi một yêu cầu trực tuyến r :

- Chèn r vào lịch trình hướng dẫn của từng taxi
- Chọn taxi có quãng đường, thời gian đi thêm là nhỏ nhất để phục vụ yêu cầu
- Những taxi không phải phục vụ yêu cầu trực tuyến vẫn di chuyển theo lịch trình hướng dẫn vì lịch trình đã tối ưu, dự đoán khả năng cao các yêu cầu sẽ xuất hiện.

Thuật toán 6: Khung chèn dự đoán - Prophet Framework

Input: weight α , workers W , requests R , and predicted requests $\tilde{R}$

Output: the unified cost $UC(W, R) \leftarrow \alpha \sum_{w \in W} D(S_w) + \sum_{r \in R^-} p_r$

- 1 guidance routes $\{S_w\}$ process the predicted requests $\tilde{R}$ by Thuật toán 5;
 - 2 set $c_r \leftarrow 0, e_r \leftarrow \infty$ for every predicted request $r \in \tilde{R}$;
 - 3 **for** each request $r \in R$ **do**
 - 4 $Cand \leftarrow$ filter the infeasible workers in W by grid index;
 - 5 $w^* \leftarrow$ not exists, $\Delta^* \leftarrow \infty$;
 - 6 **for** each candidate worker $w \in Cand$ **do**
 - 7 $\Delta \leftarrow$ prophet-insert(w, r);
 - 8 **if** $\Delta < \Delta^*$ **then**
 - 9 $w^* \leftarrow w, \Delta^* \leftarrow \Delta$;
 - 9 **if** w^* is not empty and $p_r \geq \alpha \cdot \Delta^*$ **then**
 - 10 insert r into the guidance route of w^* ;
 - 10 **else**
 - 11 reject r and $R^- \leftarrow R^- \cup \{r\}$;
 - 11 Let the workers without any assignments keep moving by the guidance routes;
-

Khung thuật toán 6 xử lý các yêu cầu trực tuyến của người dùng. Cụ thể, chúng ta tiên xử lý các lịch trình hướng dẫn $\{S_w\}$ bằng Thuật toán 5 ở dòng 1. Ở dòng 2, chúng ta thiết lập kích thước chỗ ngồi của mỗi yêu cầu dự đoán $\tilde{R}$ bằng 0 và thời hạn là ∞ , vì chúng không phải là các yêu cầu thực tế. Nếu không, một số yêu cầu thực tế không thể được chèn vào, vì các yêu cầu dự đoán đã ảnh hưởng khả năng phục vụ thật sự của các taxi. Đối với mỗi yêu cầu trực tuyến r (dòng 3), chúng ta đầu tiên lấy một tập các taxi ứng viên được ký hiệu là $Cand$ ở dòng 4. Sau đó, chúng ta áp dụng chạy thuật toán chèn

Prophet để chọn taxi w^* , taxi có thời gian di chuyển tăng ít nhất Δ^* để phục vụ yêu cầu r (dòng 5-8). Tiếp theo, chúng ta gán yêu cầu cho taxi w^* nếu hình phạt lớn hơn chi phí (tức là, $p_r \geq \alpha \cdot \Delta^*$) (dòng 9). Nếu không, chúng ta từ chối yêu cầu r và $R^- \leftarrow R^- \cup \{r\}$ (dòng 10). Các taxi không có nhiệm vụ sẽ tiếp tục di chuyển theo các lịch trình hướng dẫn (dòng 11).

Cả hai bước ở trên đều cần một thuật toán chèn, với các định nghĩa về $wait_r$ và lịch trình hướng dẫn, tôi đề xuất một thuật toán dựa trên thuật toán Linear Plain có tên là Prophet để xử lý các yêu cầu r có thông số $wait_r$.

b) Chi tiết thuật toán Prophet

Sự khác biệt lớn nhất giữa Plain và Prophet là cần xem xét thời gian chờ đợi của một taxi (waiting time). Ví dụ như nếu có một yêu cầu được dự đoán trước sẽ xuất hiện vào 7h05, tài xế có thể đến vị trí đón vào lúc 7h và đợi 5 phút cho tới khi yêu cầu xuất hiện.

Gán $wait[k]$ là thời gian đợi tại vị trí l_k của lịch trình S_w :

$$wait[k] = \begin{cases} \max(t_r - arr[k]), & l_k \text{ là điểm đón của yêu cầu} \\ 0, & l_k \text{ là điểm trả} \end{cases} \quad (1)$$

Từ đó, định nghĩa mảng arr sẽ thay đổi như sau:

$$arr[k] = \max(arr[k-1], t_r) + dis(l_{k-1}, l_k) \quad (2)$$

Trong đó, $dis(l_{k-1}, l_k)$ là thời gian di chuyển giữa hai điểm l_{k-1} và l_k và $\max(arr[k-1], t_r)$ là thời gian tài xế bắt đầu di chuyển tới vị trí tiếp theo. Tôi dùng hàm $\max$ vì có thể tài xế đến vị trí l_{k-1} trước khi yêu cầu r xuất hiện ở thời gian t_r .

Tôi định nghĩa cả tổng thời gian đợi từ vị trí l_k tới vị trí l_n là $sumWait[k]$ - tổng tiền tố (ngược từ n tới 1) của $wait$:

$$sumWait[k] = \begin{cases} 0, & k = n + 1 \\ sumWait[k+1] + wait[k], & \text{trường hợp khác} \end{cases} \quad (3)$$

Định nghĩa về thời gian đợi cũng ảnh hưởng tới slack ở được định nghĩa ở 3.6.3. Trong Prophet, một taxi có thể phải đợi ở vị trí l_{k+1} một khoảng thời gian trước khi yêu cầu tại đó xuất hiện, vì vậy taxi có thể tận dụng thời gian đợi đó để detour (đi thêm) trong khoảng địa điểm l_k và l_{k+1} . $slack[k]$ giờ được định nghĩa là:

$$slack[k] = \min\{slack[k+1], ddl[k+1] - arr[k+1]\} + wait[k+1] \quad (4)$$

Định nghĩa về mảng $Dio[j]$ ở phần Plain 3.6.3 cũng sẽ thay đổi khi thêm thời gian chờ đợi vào mỗi một yêu cầu:

$$Dio[j] = \begin{cases} \infty & \text{if } picked[j-1] > c_w - c_r \\ Dio[j-1] & \text{if } det(l_{j-1}, o_r, l_j) > slack[j-1] \\ \min\{Dio[j-1], det(l_{j-1}, o_r, l_j) - sumWait[j]\} & \text{if } otherwise \end{cases} \quad (5)$$

Mảng $Plc[j]$ tại mục 3.6.3 cũng thay đổi vì wait time:

$$Plc[j] = \begin{cases} NIL & \text{if } picked[j-1] > c_w - c_r \\ Plc[j-1] & \text{if } det(l_{j-1}, o_r, l_j) > slack[j-1] \\ Plc[j-1] & \text{if } Dio[j-1] < det(l_{j-1}, o_r, l_j) - sumWait[j] \\ j-1 & \text{if } Dio[j-1] \geq det(l_{j-1}, o_r, l_j) \end{cases} \quad (6)$$

Thời gian di chuyển tăng thêm với hai trường hợp $i = j$ và $i \neq j$ cũng thay đổi:

$$\Delta_{i,j} = \begin{cases} \max\{detOD(i) - swait[i+1], 0\} & \text{if } i = j < n \\ \max\{detD(j) + Dio[j], detD(j) - swait[j+1], 0\} & \text{if } i \neq j < n \end{cases} \quad (7)$$

với định nghĩa $detD(i = j < n)$ và $detOD(i \neq j = n)$ là:

$$detOD(j) = dis(l_j, o_r) + wait_r + dis(o_r, d_r) + dis(d_r, l_{j+1}) - dis(l_j, l_{j+1})$$

$$detD(j) = \begin{cases} dis(l_j, d_r) + dis(d_r, l_{j+1}) - dis(l_j, l_{j+1}) & \text{if } j < n, \\ dis(l_j, d_r) & \text{if } j = n. \end{cases}$$

Từ đó, thuật toán Prophet được trình bày như sau:

Thuật toán 7: Thuật toán chèn tuyến tính với dữ liệu dự đoán - Prophet

Input: a worker w with route S_w and a request r

Output: a new route S_w^* for the worker w

```
1  $S_w^* \leftarrow S_w, \Delta^* \leftarrow \infty, Dio[0] \leftarrow \infty, Plc[0] \leftarrow NIL$ 
2 Initialize  $ddl, arr, slack, picked, Plc, sumWait$ 
3 for  $j \leftarrow 0$  to  $n$  do
4    $wait_r = \max(0, t_r - (arr[j] + wait[j] + dis(l_j, o_r)))$ 
5    $\Delta_j \leftarrow \max(dis(l_j, o_r) + wait_r + dis(o_r, d_r) + dis(d_r, l_{j+1}) - dis(l_j, l_{j+1}) -$   

    $sumWait[j+1], 0)$ 
6   if  $\Delta_j < \Delta^*$  then
7      $\Delta^* \leftarrow \Delta_j$ 
8      $i^* \leftarrow j$ 
9      $j^* \leftarrow j$ 
10  if  $j > 0$  and  $picked[j] \leq c_w - c_r$  and  

    $arr[j] + wait[j] + \max(Dio[j] + sumWait[j+1], 0) + dis(l_j, d_r) \leq e_r$  and  

    $\max(Dio[j] + sumWait[j+1], 0) + det(l_j, d_r, l_{j+1}) \leq slack[j]$  then
11     $\Delta_j \leftarrow \max(Dio[j] + det(l_j, d_r, l_{j+1}), det(l_j, d_r, l_{j+1}) - sumWait[j+1], 0)$ 
12    if  $\Delta_j < \Delta^*$  then
13       $\Delta^* \leftarrow \Delta_j$ 
14       $i^* \leftarrow Plc[j]$ 
15       $j^* \leftarrow j$ 
16  if  $arr[j] + dis(l_j, d_r) > e_r$  then
17    break
18  Update  $Dio[j+1]$  and  $Plc[j+1]$  according above recipe
19 if  $\Delta^* < \infty$  then
20    $S_w^* \leftarrow \text{insert } o_r \text{ at } i^* \text{ and } d_r \text{ at } j^* \text{ in } S_w$ 
21 return  $S_w^*$ 
```

Thuật toán Prophet có một vài thay đổi so với thuật toán Plain để có thể xử lý được các yêu cầu dự đoán trước. Với trường hợp chèn cặp điểm o_r và d_r vào cùng một vị trí trong lịch trình, ta có $wait_r$ ở dòng 4 là thời gian taxi có thể phải đợi nếu yêu cầu được chèn vào vị trí j trong lịch trình. Lúc này Δ_j sẽ là thời gian tăng thêm nếu chèn yêu cầu vào vị trí j . Δ_j được tính bằng thời gian taxi đi thêm giữa hai điểm j và $j+1$. Thời gian đi thêm có thể bị triệt tiêu nếu các điểm tiếp theo của lịch trình, taxi phải đợi yêu cầu xuất hiện, ta có thể coi như đã "tận dụng" thời gian đợi này để giảm đi thời gian di chuyển thêm của taxi, vì vậy Δ_j cần phải trừ đi tổng thời gian đợi của các điểm còn lại của lịch trình - $sumWait[j+1]$.

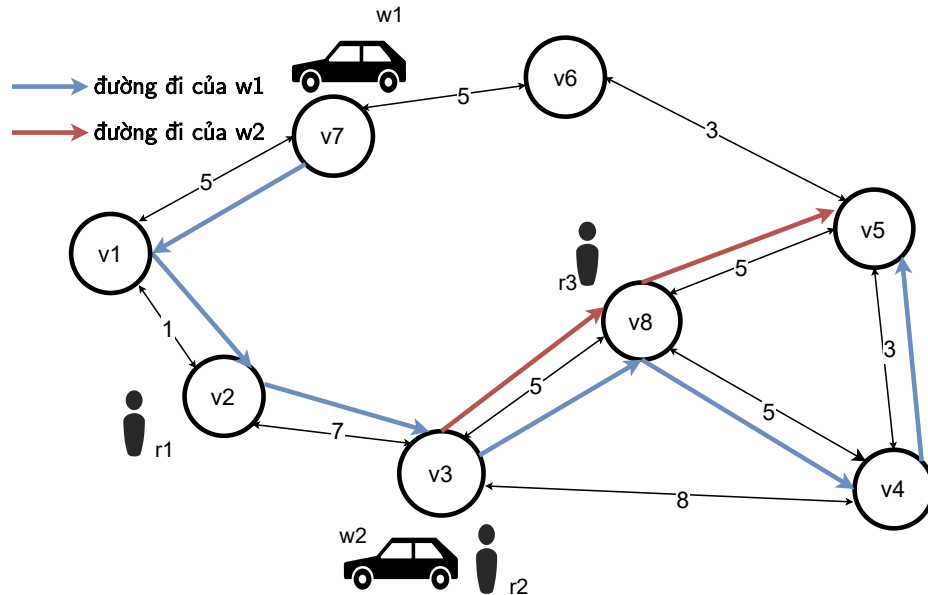
Với các trường hợp chen cặp điểm o_r và d_r khác vị trí nhau, tôi vẫn chỉ xét điểm chen d_r nhưng có một vài thay đổi. Các mảng Dio, Plc đã được thay đổi và điều kiện để có thể chen d_r vào lịch trình ở dòng 7 trở thành thời gian taxi đến điểm j ($arr[j]$) cộng thời gian taxi chờ ở j ($wait[j]$) cộng quãng đường tăng thêm tối thiểu khi thêm o_r vào vị trí $Plc[j]$ cộng tổng thời gian chờ từ $j+1$ tới n ($sumWait[j+1]$) cộng thời gian di chuyển từ j đến d_r ($dis(l_j, d_r)$) phải nhỏ hơn thời gian deadline của yêu cầu e_r . Điều kiện thứ hai là thời gian tăng thêm tối thiểu khi chen o_r vào vị trí $Plc[j]$ cộng thời gian chờ từ $j+1$ tới n ($sumWait[j+1]$) và thời gian tăng thêm khi chen d_r vào giữa j và $j+1$ ($det(l_j, d_r, l_{j+1})$) phải nhỏ hơn thời gian detour tối đa ($slack[j]$).

c) Phân tích độ phức tạp thuật toán

Khởi tạo các mảng $ddl, arr, slack, picked, wait, sumWait$ ở dòng 2 tốn thời gian $O(n)$. Tiếp theo ở dòng thứ 3, sử dụng một vòng lặp và từ dòng 4 - 10 trong vòng lặp chỉ tốn thời gian $O(1)$. Nên tổng chi phí của vòng lặp chỉ là $O(n)$. Cuối cùng ở dòng 11, như hai thuật toán trước đều tốn thời gian $O(n)$ để chen o_r vào điểm i và d_r vào điểm j . Như vậy tổng chi phí của thuật toán này về thời gian là $O(n)$ và chi phí cho bộ nhớ là $O(n)$.

3.6.5 Ví dụ khung thuật toán chen dự đoán

Có một đồ thị thể hiện mạng lưới giao thông như sau:



Hình 3.6: [Ví dụ] Đồ thị mạng lưới giao thông

Và một số các yêu cầu mới ở bảng 3.6 với: t_r là thời gian xuất hiện, e_r là deadline

Yêu cầu	t_r	e_r	o_r	d_r	p_r	c_r
r_1	5	28	v_2	v_4	20	1
r_2	10	31	v_3	v_5	10	1
r_3	11	26	v_8	v_5	9	1

Bảng 3.6: Bảng thông tin các yêu cầu động mới xuất hiện r_1 - r_3

phải hoàn thành yêu cầu, o_r là vị trí đón khách, d_r là vị trí trả khách, p_r là hệ số phạt nếu không phục vụ, c_r là số lượng người trong một yêu cầu.

Để đơn giản, coi tập yêu cầu tiên đoán là tập yêu cầu mới, từ đó có $\tilde{R} = R = \{r_1, r_2, r_3\}$. Đầu tiên, hệ thống sẽ dùng thuật toán 5 để đưa các yêu cầu dự đoán vào lịch trình hướng dẫn. Với yêu cầu dự đoán đầu tiên r_1 , tạo một tài xế ảo $\tilde{w}_1$ đứng ở vị trí ban đầu là v_2 (vị trí đón của r_1) với dung lượng chỗ ngồi là 4 ở dòng 6, khi này $\tilde{W}$ đang trống. $\tilde{w}_1$ có thể phục vụ r_1 theo lịch trình hướng dẫn $\langle (v_2, 0), (v_2, 5), (v_4, 0) \rangle$. Ví dụ, $\tilde{w}_1$ sẽ đợi tại vị trí $l_1 = v_2$ trong thời gian 5 cho đến khi r_1 xuất hiện. Khi yêu cầu dự đoán tiếp theo r_2 được lập tại dòng 2, dịch vụ sử dụng phương pháp Prophet để đưa nó vào lịch trình hướng dẫn của $\tilde{w}_1$. Chi tiết thuật toán sẽ được trình bày trong đoạn tiếp theo, và hệ thống có thể tính được $\tilde{w}^* = \tilde{w}_1, \Delta^* = 8$ (dòng 4) và lịch trình hướng dẫn mới là $\langle (v_2, 0), (v_2, 5), (v_3, 0), (v_4, 0), (v_5, 0) \rangle$ (dòng 5). Tương tự, khi yêu cầu dự đoán cuối cùng r_3 được lập tại dòng 2, có thể tính được $\tilde{w}^* = \tilde{w}_1, \Delta^* = 2$ và lịch trình hướng dẫn mới là $\langle (v_2, 0), (v_2, 5), (v_3, 0), (v_8, 0), (v_4, 0), (v_5, 0), (v_5, 0) \rangle$. Tại dòng 9, đưa $\tilde{w}_1$ ra khỏi heap Q và kiểm tra xem các tài xế thực ($w_1 - w_2$) có thể phục vụ các yêu cầu dự đoán (tức là $r_1 - r_3$) trên lịch trình hướng dẫn của $\tilde{w}_1$ hay không. Ví dụ, lịch trình hướng dẫn khả thi cho $\tilde{w}_1$ là $\langle (v_7(o_{\tilde{w}_1}), 0), (v_2, 0), (v_3, 0), (v_8, 0), (v_4, 0), (v_5, 0), (v_5, 0) \rangle$. Lịch trình hướng dẫn của $\tilde{w}_2$ trống.

Từ việc xử lý các yêu cầu dự đoán, có 2 lịch trình hướng dẫn là

$$S_{\tilde{w}_1} = \langle (v_7, 0), (v_2, 0), (v_3, 0), (v_8, 0), (v_4, 0), (v_5, 0), (v_5, 0) \rangle \text{ và } S_{\tilde{w}_2} = \emptyset$$

Khi yêu cầu trực tuyến đầu tiên r_1 xuất hiện tại thời điểm 5, w_1 di chuyển đến v_1 và w_2 ở lại v_3 . Giả sử các tài xế ảo khả thi là $\{w_1, w_2\}$ (dòng 4). Ở dòng 7, thuật toán tính được thời gian di chuyển tăng thêm tối thiểu $\Delta = 0$ và 17 để chèn yêu cầu mới vào w_1 và w_2 tương ứng, trong đó chi tiết tính toán sẽ được trình bày trong ví dụ thuật toán Prophet ở phần sau. Do đó, có $w^* = w_1$ và $\Delta^* = 0$ ở dòng 9. Vì mức phạt của $r_1(p_{r_1} = 20)$ lớn hơn chi phí di chuyển tăng thêm ($\alpha \cdot \Delta^* = 0$), thuật toán quyết định phục vụ yêu cầu r_1 và gán nó cho w_1 . Thuật toán cũng cập nhật tuyến hướng dẫn hiện tại của tài xế, tức là: $\langle (v_1, 0), (v_2(o_{r_1}), 0), (v_2, 0), (v_3, 0), (v_8, 0), (v_4(d_{r_1}), 0), (v_4, 0), (v_5, 0), (v_5, 0) \rangle$. Tương tự, khi các yêu cầu r_2 và r_3 xuất hiện, chúng cũng sẽ được gán cho w_1 với thời gian di

chuyển tăng thêm tối thiểu $\Delta^* = 0$. Theo đó, lịch trình di chuyển của w_1 được đánh dấu bằng các mũi tên xanh trong hình 3.6 và w_2 luôn ở vị trí ban đầu v_3 . Do đó, chi phí là $1 \times ((6 + 7 + 5 + 5 + 3) + 0) + 0 = 26$.

Để tính toán chi phí di chuyển tăng thêm tối thiểu, và tìm được vị trí chèn o_r và d_r vào lịch trình ta sẽ đi qua thuật toán Prophet với mỗi một tài xế khả thi:

Một yêu cầu dự đoán r_2 được chèn vào lịch trình $\langle (v_2, 0), (v_2, 5), (v_4, 0) \rangle$. Chúng ta có thể sử dụng Thuật toán 7 để tính toán thời gian di chuyển tăng thêm tối thiểu. Các mảng ở dòng 2 của Thuật toán 7 được khởi tạo như trong Bảng 3.7.

k	$wait[k]$	$swait[k]$	$ddl[k]$	$arr[k]$	$picked[k]$	$slack[k]$	$Dio[k]$	$Plc[k]$
0	0	5	0	0	0	18	∞	không tồn tại
1	5	5	18	0	1	13	12	0
2	0	0	28	15	0	∞	5	1

Bảng 3.7: Bảng khởi tạo các mảng cho ví dụ 3.6.5

Cụ thể, thời gian chờ $wait[k]$ được tính bằng Phương trình (1) ở phần 3.6.4, trong ví dụ: $wait[1] = \max\{t_{r_1} - arr[1], 0\} = 5$.

Theo Phương trình (3), thuật toán có thể tính $swait[k]$, trong ví dụ: $swait[1] = wait[1] + swait[2] = 5$.

Thuật toán khởi tạo $ddl[\cdot]$ và $pick[\cdot]$ như trong thuật toán Plain:

Theo Phương trình (2), thuật toán khởi tạo mảng $arr[k]$, trong ví dụ: $arr[2] = \max\{arr[1], t_{r_1}\} + dis(v_2, v_4) = \max\{0, 5\} + 10 = 15$.

Theo Phương trình (4), chúng ta có $slack[1] = wait[2] + \min\{ddl[2] - arr[2], slack[2]\} = 0 + \min\{28 - 15, \infty\} = 13$, $slack[0] = wait[1] + \min\{ddl[1] - arr[1], slack[1]\} = 5 + \min\{18 - 0, 13\} = 18$.

Ngoài ra, có $Dio[0] = \infty$, $Plc[0] = \{\text{không tồn tại}\}$, vì không thể tồn tại i khả thi cho $j = 0$ trong trường hợp tổng quát $i < j$ như được chỉ ra trong Hình 3.5.

Sử dụng các Phương trình (5) và (6), thuật toán có:

$$Dio[1] = \min\{Dio[0], detO(0) - swait[1]\} = \min\{\infty, 17 - 5\} = 12$$

$$Plc[1] = 1 - 1 = 0$$

vì $detO(0) = dis(v_2, o_{r_2}) + wait_{r_2} + dis(o_{r_2}, v_2) - dis(v_2, v_2) = 7 + 3 + 7 - 0 = 17 < slack[0]$ (trường hợp cuối cùng của Phương trình (5))

và

$$Dio[0] > detO(0) - swait[1]$$

với $wait_{r_2} = \max\{0, t_{r_2} - (arr[0] + wait[0] + dis(v_2, or_2))\} = \max\{0, 10 - (0 + 0 + 7)\} = 3$ (trường hợp cuối cùng của Phương trình (6)).

Tương tự, thuật toán cũng có

$$Dio[2] = \min\{Dio[1], detO(1) - swait[2]\} = \min\{12, 5 - 0\} = 5$$

$$plc[2] = 2 - 1 = 1$$

vì $detO(1) = dis(v_2, or_2) + wait_{r_2} + dis(or_2, v_4) - dis(v_2, v_4) = 7 + 0 + 8 - 10 = 5 < slack[1]$ (trường hợp cuối cùng của Phương trình (5))

và

$$dio[1] > detO(1) - swait[2]$$

trong đó $wait_{r_2} = \max\{0, t_{r_2} - (arr[1] + wait[1] + dis(v_2, or_2))\} = \max\{0, 10 - (0 + 5 + 7)\} = 0$ (trường hợp cuối cùng của Phương trình (6)).

Trong các dòng 4-6, ta xử lý **trường hợp đặc biệt** $i = j$. Khi $j = 0$, ta thấy dòng 5 vi phạm, vì:

$$\begin{aligned} detOD(0) &= dis(v_2, or_2) + wait_{r_2} + dis(or_2, dr_2) + dis(dr_2, v_2) - dis(v_2, v_2) \\ &= 7 + 3 + 10 + 10 - 0 = 30 > slack[0] \end{aligned}$$

với $wait_{r_2} = \max\{0, t_{r_2} - (arr[0] + wait[0] + dis(v_2, or_2))\} = \max\{0, 10 - (0 + 0 + 7)\} = 3$.

Ta lại có:

$$\begin{aligned} detOD(1) &= dis(v_2, or_2) + wait_{r_2} + dis(or_2, dr_2) + dis(dr_2, v_4) - dis(v_2, v_4) \\ &= 7 + 0 + 10 + 3 - 10 = 10 < slack[1] \end{aligned}$$

trong đó $wait_{r_2} = \max\{0, t_{r_2} - (arr[1] + wait[1] + dis(v_2, or_2))\} = \max\{0, 10 - (0 + 5 + 7)\} = 0$ như phương trình (7).

Ở dòng 5, ta có:

$$arr[1] + wait[1] + dis(v_2, or_2) + wait_{r_2} + dis(or_2, dr_2) = 0 + 5 + 7 + 0 + 10 = 22 < e_{r_2} = 31$$

Ở dòng 6, ta tính toán thời gian di chuyển tăng thêm $\Delta_{i,j} = \max\{detOD(1) - swait[2], 0\} = \max\{10 - 0, 0\} = 10$. Khi các lần lặp trong các dòng 4-6 dừng lại, ta có $\Delta = \Delta_{i,j} = 10$, $i^* = 1$, $j^* = 1$.

Trong các dòng 7-11, ta xử lý **trường hợp tổng quát** $i < j$:

Khi $j = 1$, vì phạm ở dòng 7, vì:

$$detD(1) + \max\{dio[1] + swait[2], 0\} = 3 + \max\{12 + 0, 0\} = 15 > slack[1]$$

trong đó $detD(1) = dis(v_2, dr_2) + dis(dr_2, v_4) - dis(v_2, v_4) = 10 + 3 - 10 = 3$.

Khi $j = 2$, dòng 7 thoả mãn vì $slack[2] = \infty$. Ta có:

$$arr[2] + wait[2] + \max\{dio[2] + swait[3], 0\} + dis(v_4, dr_2) = 15 + 0 + \max\{5 + 0, 0\} + 3 = 23 < e_{r_2} = 31$$

Ở dòng 9, ta tính toán thời gian di chuyển tăng thêm Δ_2 . Có:

$$\Delta_2 = \max\{detD(2) + dio[2], detD(2) - swait[3], 0\} = \max\{3 + 5, 3 - 0, 0\} = 8$$

trong đó $detD(2) = dis(v_4, dr_2) = 3$. Vì $\Delta_2 < \Delta$, ta cập nhật $\Delta = \Delta_2 = 8$, $i^* = plc[2] = 1$, $j^* = j = 2$ ở dòng 9. Cuối cùng, ta có thể thu được một lịch trình mới $((v_2, 0), (v_2, 5), (v_3(or_2), 0), (v_4, 0), (v_5(dr_2), 0))$ ở dòng 12.

Vì vậy, từ ví dụ trên ta có thể thấy được cách hoạt động của thuật toán Prophet. Các tài xế có lịch trình hướng dẫn sẽ được định hướng đi tới những chỗ có nhiều yêu cầu theo dự đoán. Khi đó, mặc dù mất thêm thời gian chờ đợi nhưng ta tận dụng được sức chứa của một tài xế.

3.6.6 Thuật toán Prune Prophet

a) Ý tưởng thuật toán

Thuật toán này được tôi cải tiến từ thuật toán Prophet ở trên. Ý tưởng ở đây là tôi không bắt đầu duyệt các vị trí chèn d_r từ đầu tới cuối mà tôi sẽ loại bỏ đi một số điểm dựa trên việc tìm ra vị trí đầu tiên và cuối cùng có thể chèn d_r thông qua điều kiện về thời gian.

b) Chi tiết thuật toán

Thuật toán 8: Tìm vị trí đầu tiên và cuối cùng để chèn d_r

Input: a worker w with route S_w and a request r

Output: starting index to insert o_r

```
1  $beginJ \leftarrow -1$  Vị trí đầu tiên có thể chèn  $d_r$ 
2  $start \leftarrow 0, end \leftarrow schedule.size() - 1, mid$ 
3 while  $start \leq end$  do
4    $mid \leftarrow start + (end - start)/2$ 
5   if  $deadlineTime_{mid} > t_r$  then
6      $end \leftarrow mid - 1$ 
7      $beginJ \leftarrow mid$ 
8   else  $start \leftarrow mid + 1$ 
7  $endJ \leftarrow -1$  Vị trí cuối cùng có thể chèn  $d_r$ 
8  $start \leftarrow 0, end \leftarrow schedule.size() - 1, mid$ 
9 while  $start \leq end$  do
10   $mid \leftarrow start + (end - start)/2$ 
11  if  $arr[mid] \geq e_r$  then
12     $end \leftarrow mid - 1$ 
13     $endJ \leftarrow mid$ 
14  else  $start \leftarrow mid + 1$ 
13 return  $beginJ$  and  $endJ$ 
```

Thuật toán dựa trên hai nhận định sau:

i) Nếu deadline của yêu cầu tại l_k sớm hơn với thời gian xuất hiện của yêu cầu mới r , tất cả các điểm $j < k$ có thể được loại bỏ $\Rightarrow$ tìm ra điều đầu tiên để chèn d_r .

ii) Nếu thời điểm xe taxi tới l_k lớn hơn thời gian deadline của yêu cầu mới r , tất cả các điểm $j \geq k$ có thể được bỏ $\Rightarrow$ tìm ra điểm cuối cùng phù hợp để chèn d_r .

Sau đó, điểm bắt đầu và kết thúc của vị trí chèn điểm cuối d_r được đưa vào dòng 3 thuật toán Prophet-Insertion 7.

c) Phân tích độ phức tạp thuật toán

Thuật toán này có độ phức tạp là $O(n)$ giống với thuật toán Linear Insertion và Linear Prophet. Nhưng thực tế là nhanh hơn vì thuật toán này đã cắt tĩa đi các trường hợp không thể chèn d_r . Mặc dù việc cắt tĩa này là tham lam và điều này làm cho kết quả tìm ra vị trí chèn cặp (o_r, d_r) không còn là tối ưu nhất nữa nhưng đổi lại hiệu năng về thời gian của thuật toán lại được cải thiện hơn.

Chương 4. MÔI TRƯỜNG THỰC NGHIỆM

Chương này giới thiệu về giao diện, chức năng, cơ sở dữ liệu, API, ... của sản phẩm. Ngoài ra chương này tôi cũng mô tả phần xây dựng thực nghiệm để đánh giá hiệu năng giữa các thuật toán trong thực tế.

4.1 Nền tảng môi trường

Hệ thống được triển khai theo mô hình client - server, gồm ứng dụng web để quản lý taxi và tuyến đường đi chung.

Website quản lý hệ thống: Giao diện(front-end) phát triển trên nền tảng web sử dụng ngôn ngữ Javascript và framework NextJS với môi trường thực thi là Node.js phiên bản 22.0.1. Back-end sử dụng ngôn ngữ Java (JDK17) với framework Spring Boot.

Server: MacOS - Intel Core I7 9750H - 16GB RAM

Database: sử dụng cơ sở dữ liệu quan hệ MySQL.

Maps: sử dụng HERE Maps API, được XanhSM sử dụng cho dịch vụ gọi xe.

4.2 Xây dựng dữ liệu

4.2.1 Dữ liệu nhóm đi chung xe

Để phục vụ cho mục đích đánh giá hiệu năng của thuật toán Prophet. Tôi sẽ tiến xử lý dữ liệu dự đoán về yêu cầu đi chung xe, dựa trên cơ sở dữ liệu cũ của hệ thống. Trong thực tế, ta sẽ dựa vào lượng dữ liệu khổng lồ về yêu cầu đi xe cá nhân của người dùng ứng dụng, đưa vào trong các mô hình học máy, yêu cầu dự đoán về nhu cầu đi xe chung sẽ được tạo ra. Sau đó trải qua thuật toán 5 để hình thành nên các lịch trình hướng dẫn như dưới đây với độ chính xác dự đoán là 25%.

	id	driver_id	type	
1	1366	1	0	
2	1367	2	0	
3	1368	3	0	
4	1369	4	0	
5	1370	5	0	
6	1371	6	0	
7	1372	7	0	
8	1373	8	0	
9	1374	9	0	

Hình 4.1: Dữ liệu các lịch trình hướng dẫn taxi di chuyển tới các nơi có nhiều yêu cầu

Hình dưới là dữ liệu chi tiết lịch trình của một nhóm đi chung. Với mỗi nhóm đi chung sẽ bao gồm tập hợp các điểm đón, điểm trả. Mỗi điểm như vậy tương ứng với một bản ghi trong bảng ở hình bên dưới.

id	capacity	expect	group	lat	ln	locati	passenger_id	schedule
1	12443	3	6.83	1366	21.002963	105.779002	1	1
2	12444	3	9.3149	1366	21.029192	105.806268	2	1
3	12445	3	9.7636	1366	21.034341	105.787125	1	1
4	12446	3	10.308100000000000...	1366	21.007232	105.780993	2	1
5	12447	3	9.921600000000000...	1366	21.006124	105.822254	1	1
6	12448	3	10.098300000000000...	1366	20.994623	105.800259	2	1
7	12451	3	9.433599999999998	1366	21.0379881...	105.81508...	1	1
8	12452	3	9.600699999999998	1366	21.0354363...	105.80431...	2	1
9	12487	3	9.2402	1366	21.027558	105.797203	1	1
10	12488	3	10.441300000000000...	1366	20.997724	105.790622	2	1
11	12499	1	8.0599	1366	21.012698	105.789414	1	1
12	12500	3	9.6535	1366	21.035939	105.79623	2	1
13	12513	1	7.185000000000000...	1366	20.9990346...	105.79810...	1	1
14	12514	3	8.176100000000000...	1366	21.0282387...	105.77869...	2	1
15	12521	-4	7.316000000000000...	1366	21.003809	105.79544	1	1
16	12522	1	7.891100000000000...	1366	20.998982	105.809183	2	1
17	12539	-9	7.491200000000000...	1366	21.010005	105.789149	1	1
18	12540	-4	7.728000000000000...	1366	21.018726	105.822464	2	1
19	12697	0	9.060200000000000...	1366	21.040785	105.781773	1	1

Hình 4.2: Dữ liệu lịch trình của một nhóm đi chung

4.2.2 Dữ liệu yêu cầu đi chung xe trực tuyến

Tôi sinh các yêu cầu đi chung xe trực tuyến ngẫu nhiên. Các yêu cầu đi chung xe trực tuyến có dạng như hình bên dưới.

```
{
  "pickUpTime": 7.07,
  "capacity": 1,
  "latOrigin": 21.03880193405791,
  "lngOrigin": 105.7805796843511,
  "latDestination": 21.02636443516133,
  "lngDestination": 105.7785889212281,
  "addressStart": "16 Phạm Văn Đồng, Mai Dịch, Cầu Giấy, Hà Nội, Vietnam",
  "addressEnd": "40 Phạm Hùng, Mỹ Đình, Từ Liêm, Hà Nội, Vietnam",
  "length": 1.5
},
{
  "pickUpTime": 7.1,
  "capacity": 1,
  "latOrigin": 21.046027218991476,
  "lngOrigin": 105.78884569552218,
  "latDestination": 21.02499070127436,
  "lngDestination": 105.84316056509769,
  "addressStart": "A26 Hoàng Quốc Việt, Nghĩa Tân, Cầu Giấy, Hà Nội, Vietnam",
  "addressEnd": "Pacific Place Building, 83B P. Lý Thường Kiệt, Cửa Nam, Hoàn Kiếm, Hà Nội 100000, Vietnam",
  "length": 7.2
},
{
  "pickUpTime": 7.11,
  "capacity": 2,
  "latOrigin": 20.998206079393842,
  "lngOrigin": 105.81329203048001,
  "latDestination": 21.00778305950893,
  "lngDestination": 105.82371909896521,
  "addressStart": "227 Đ. Nguyễn Trãi, Thượng Đình, Thanh Xuân, Hà Nội, Vietnam",
```

Hình 4.3: Dữ liệu yêu cầu đi chung xe trực tuyến

4.2.3 Dữ liệu yêu cầu đi chung xe dự đoán

Từ các yêu cầu đi chung xe trực tuyến, tôi sinh ra các yêu cầu đi chung xe dự đoán dựa trên thời gian - địa điểm (mô phỏng đầu ra của một mô hình học máy ví dụ như [8])

```
{
  "showTime": 6.8,
  "capacity": 1,
  "latOrigin": 21.021398791590123,
  "lngOrigin": 105.80096514892128,
  "latDestination": 21.018009858860477,
  "lngDestination": 105.8073083743843,
  "addressStart": "918 Đ. Láng, Yên Hoà, Đống Đa, Hà Nội 100000, Vietnam",
  "addressEnd": "99 Đ. Nguyễn Chí Thanh, Láng Hạ, Đống Đa, Hà Nội 11500, Vietnam",
  "length": 1.4
},
{
  "showTime": 7,
  "capacity": 1,
  "latOrigin": 21.03360971973142,
  "lngOrigin": 105.77980866171949,
  "latDestination": 21.02340743563475,
  "lngDestination": 105.7905529674234,
  "addressStart": "8 Phạm Hùng, Mỹ Trì, Nam Từ Liêm, Hà Nội, Vietnam",
  "addressEnd": "226 P. Trung Kính, Yên Hoà, Cầu Giấy, Hà Nội, Vietnam",
  "length": 2.8
},
{
  "showTime": 7,
  "capacity": 1,
  "latOrigin": 21.023777680007235,
  "lngOrigin": 105.81749530270386,
  "latDestination": 21.02933692388695,
  "lngDestination": 105.8271188985553,
  "addressStart": "626 Đ. La Thành, Thành Công, Ba Đình, Hà Nội, Vietnam",
  "addressEnd": "2RHG+MVH, Chợ Dừa, Đống Đa, Hà Nội, Vietnam",
  "length": 2.8
}
```

Hình 4.4: Dữ liệu yêu cầu đi chung xe trực tuyến

4.3 Xây dựng dữ liệu bản đồ lưới

Hệ thống hiện tại hướng đến phục vụ ở khu vực Hà Nội, nên tôi thiết kế bản đồ Hà Nội thành một bản đồ lưới như sau:

- + Bản đồ lưới được giới hạn bởi bốn góc, trong đó:
 - Kinh độ: nằm trong khoảng từ $20^{\circ} 59' 24''$ ($= 20.99$) đến $21^{\circ} 3' 0''$ ($= 21.05$)
 - Vĩ độ: nằm trong khoảng từ $105^{\circ} 46' 12''$ ($= 105.77$) đến $105^{\circ} 51' 0''$ ($= 105.85$)
- + Mỗi ô trong bản đồ lưới là ô vuông có kích thước 250m x 250m. Như vậy tôi sẽ chia được thành $32 \times 64 = 768$ ô và $768 \times 768 = 589.824$ cạnh.
- + Dưới đây là mã giả ánh xạ một điểm trên bản đồ thành ô trong bản đồ lưới

Thuật toán 9: Ánh xạ một điểm lên lưới bản đồ

Input: a point with *latitude* and *longitude*

Output: cell id when mapping a point to a grid

```
1 if latitude < START_LATITUDE or latitude > END_LATITUDE or  
   longitude < START_LONGITUDE or longitude > END_LONGITUDE  
   then  
   | return-1  
2 indexLatitude ← (latitude − START_LATITUDE)/LENGTH_REGION  
   indexLongitude ←  
     (longitude − START_LONGITUDE)/LENGTH_REGION  
3 return  
   indexLatitude * NUMBER_CELL_LONGITUDE + indexLongitude + 1
```

Với *START_LATITUDE*, *END_LATITUDE*, *START_LONGITUDE*, *END_LONGITUDE*, *NUMBER_CELL_LONGITUDE*, *LENGTH_REGION* là các hằng số lần lượt là mốc kinh độ bắt đầu (= 20.99), mốc kinh độ kết thúc (= 21.05), mốc vĩ độ bắt đầu (= 105.77), mốc vĩ độ kết thúc (= 105.85), số lượng khoảng cách được chia theo vĩ độ (= 32) và độ dài mỗi khoảng tương ứng với độ dài mỗi cạnh của một ô vuông trong lưới (= 0.0025).

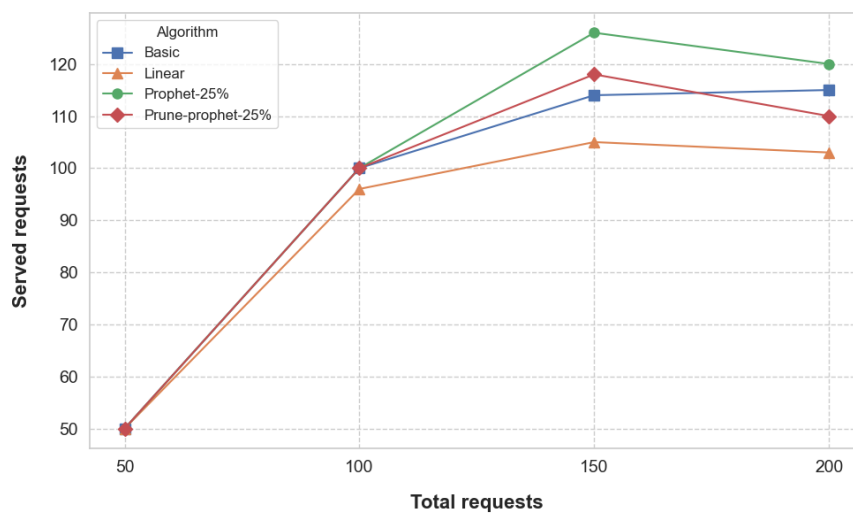
4.4 Đánh giá hiệu năng thuật toán

Trong tiểu mục này, tôi thực hiện đánh giá hiệu năng giữa bốn thuật toán Basic Plain, Linear Plain, Prophet, Prune Prophet. Bộ dữ liệu được sử dụng gồm tổng 200 yêu cầu đi chung xe và 21 xe taxi với sức chứa khác nhau. Nếu không có nhóm đi chung phù hợp để phục vụ một yêu cầu đi chung xe trực tuyến thì một taxi chưa có lịch trình sẽ được sử dụng để phục vụ yêu cầu đó. Mục đích chính của hệ thống đi chung xe là giải quyết được vấn đề tắc nghẽn giao thông, đặc biệt là thời gian 7 - 8h là lúc mọi người đi học, đi làm. Nên dữ liệu về 200 yêu cầu hầu hết có điểm đến là các trường học, công ty với thời gian trong khoảng từ 7:00 - 8:30 sáng. Hệ thống sử dụng tiêu chí tổng thời gian phản hồi cho 200 yêu cầu để so sánh giữa bốn thuật toán chèn một cặp điểm đi, điểm đón của yêu cầu. Ngoài ra, tôi cũng so sánh cả bốn thuật toán với phương pháp không đi chung xe hiện nay (nghĩa là mỗi người đi một xe hoặc tài xế chỉ chở 1 hành khách duy nhất) dựa trên tiêu chí số lượng xe cần thiết để phục vụ 200 yêu cầu. Lí do tôi chọn 200 yêu cầu và 21 taxi trong khoảng thời gian 1h đó chính là muốn thử nghiệm khả năng phục vụ của thuật toán Prophet so với các thuật toán khác trên phương diện số xe taxi có hạn.

4.4.1 So sánh bốn thuật toán đi chung xe với không đi chung xe

Phần thực nghiệm này so sánh số lượng taxi cần sử dụng để phục vụ được tất cả yêu cầu đi chung xe trực tuyến. Trong mỗi trường hợp như thế số lượng yêu cầu đi chung xe trực tuyến sẽ tăng từ 50 lên 100, 150 và cuối cùng là 200 yêu cầu. Với các thuật toán có sử dụng dữ liệu, tôi đưa ra độ chính xác của dự đoán là 25%. Mặc dù hiện nay đã có những bài báo, công trình nghiên cứu cho ra kết quả tốt hơn nhưng 25% chính xác đã có thể đánh giá được hiệu quả của thuật toán.

Tỉ lệ phục vụ với một lượng xe giới hạn là 20

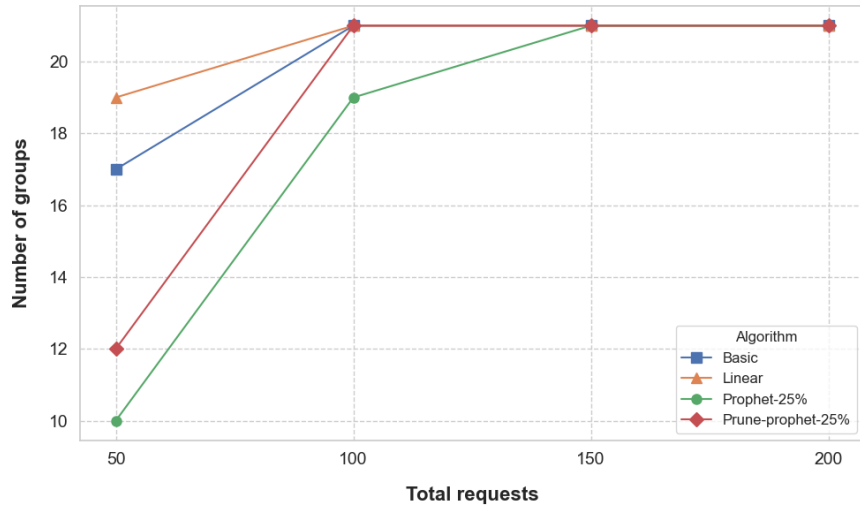


Hình 4.5: Số lượng yêu cầu được phục vụ

Kết quả thực nghiệm cho thấy cả bốn thuật toán đều phục vụ tốt tổng số yêu cầu nằm trong khoảng 120. Với số lượng yêu cầu lớn hơn 150, số lượng xe 20 là không đủ để phục vụ hết.

Thuật toán Prophet cho ra kết quả tốt nhất với số lượng xe bị giới hạn như trường hợp này, cùng với đó là thuật toán Prune-Prophet khi đã cắt giảm đi vài trường hợp chèn mà vẫn giữ được tỉ lệ phục vụ cao.

Số lượng taxi cần dùng khi có nhiều nhất 20 xe

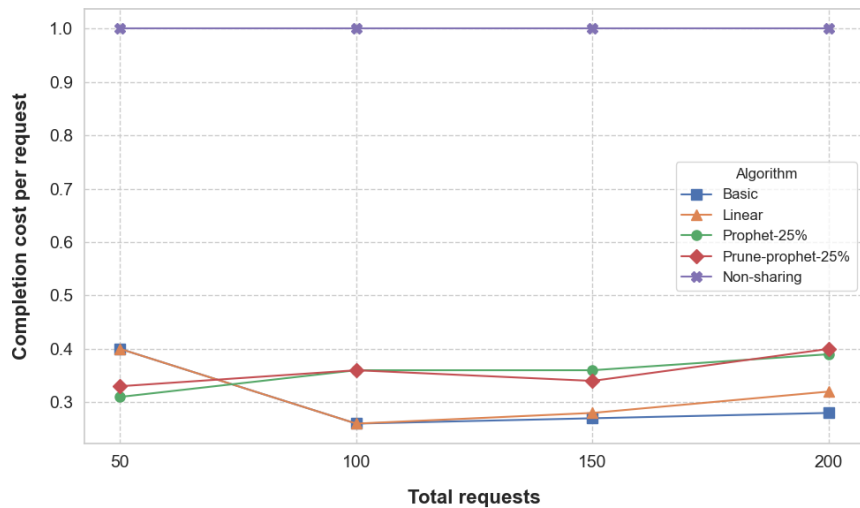


Hình 4.6: Số lượng taxi cần sử dụng để phục vụ yêu cầu đi chung xe trực tuyến

Biểu đồ thực nghiệm cho thấy với thuật toán Prophet, số lượng taxi cần sử dụng thấp hơn rất nhiều so với các thuật toán chèn thông thường, chỉ 9 xe taxi cần sử dụng với 50 yêu cầu so với 17 taxi của thuật toán Linear Plain mà thời gian chạy không chậm hơn.

Cùng với đó, số lượng taxi của thuật toán Prune-Prophet cũng cho thấy thuật toán này kém chính xác hơn Prophet.

Nếu so sánh cả bốn thuật toán với phương án không chia sẻ chuyến đi, đặt xe cá nhân thì đây là một kết quả rất tốt. Với 100 yêu cầu, thuật toán Prophet chỉ cần 18 phương tiện phục vụ thay vì 100 với các hệ thống hiện nay.



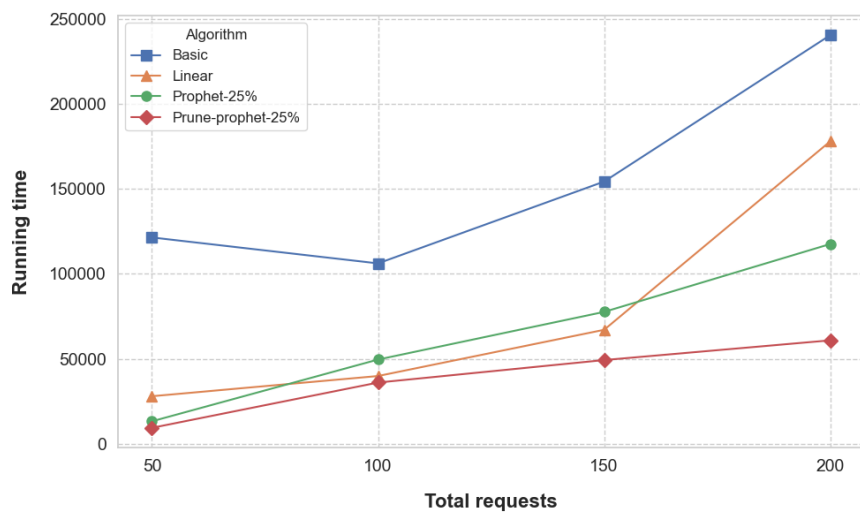
Hình 4.7: Chi phí mỗi một yêu cầu được phục vụ

Với cách di chuyển bằng phương tiện cá nhân/đặt xe riêng, chi phí cho mỗi yêu cầu

bình quân là 1. Cả bốn thuật toán đều giảm được một khoản chi phí lớn về di chuyển - gần 50% so với phương án không đi chung. Chi phí ở đây được tính là quãng đường - thời gian di chuyển trên đường, những quãng đường đi chung sẽ không được tính nhiều lần. Với cách tính như vậy, các thuật toán dự đoán lịch trình sẽ tốn nhiều chi phí hơn vì hoạt động tự điều phối xe ra các vùng có nhiều yêu cầu, thay vì chỉ đứng yên đợi yêu cầu tới.

4.4.2 So sánh hiệu năng giữa ba thuật toán

Phần thực nghiệm này so sánh tổng thời gian phản hồi cho tất cả yêu cầu đi chung xe trực tuyến.



Hình 4.8: Thời gian tổng để phục vụ 200 yêu cầu đi chung xe (đơn vị: s)

Nhìn vào biểu đồ có thể thấy hai thuật toán Prophet có thời gian xử lý ít hơn nhất nhiều so với thuật toán Basic Plain. Cùng với đó, thuật toán Linear Plain khi số lượng yêu cầu nhiều cũng trở nên chậm chạp so với Prophet vì thuật toán Prophet đã tối ưu các nhóm đi chung dựa trên lịch sử đặt xe của từng khu vực - thời gian, vì vậy khi chen yêu cầu mới vào lịch trình, Prophet thể hiện tốt hơn gần gấp 2 lần.

Với việc loại bỏ đi vài trường hợp không thỏa mãn, thuật toán Prune Prophet có thời gian xử lý ngắn hơn nhiều so với thuật toán Prophet nếu lượng yêu cầu là lớn. Kết quả này hứa hẹn việc xử lý nhiều yêu cầu đặt xe ở hệ thống thực tế, khi mà trải nghiệm người dùng cần đặt lên hàng đầu nên thời gian xử lý phải cực nhanh.

4.5 Danh sách API

Dưới đây là một số API chính của hệ thống. Vì hệ thống có sử dụng nhiều thuật toán nên thiết kế API sẽ khác với các thiết kế thông thường. Ví dụ như khi người dùng đặt

xe thì app có thể chỉ gọi lên API /insert. Nhưng ở đây cần chia làm 3 API để phân định rạch ròi giữa các thuật toán.

4.5.1 API basicInsert

Chức năng: Tìm kiếm và chèn yêu cầu đi chung xe trực tuyến vào lịch trình taxi phù hợp theo thuật toán Basic Plain

URI: /insert/basic

Method: POST

4.5.2 API linearInsert

Chức năng: Tìm kiếm và chèn yêu cầu đi chung xe trực tuyến vào lịch trình taxi phù hợp theo thuật toán Linear Plain

URI: /insert/linear

Method: POST

4.5.3 API preprocessPredicted

Chức năng: Xử lý dữ liệu lịch sử đi xe và tạo ra các lịch trình hướng dẫn cho các taxi hiện có để phục vụ cho thuật toán Prophet

URI: /schedule/pre-process

Method: POST

4.5.4 API prophetInsert

Chức năng: Tìm kiếm và chèn yêu cầu đi chung xe vào lịch trình hướng dẫn của taxi phù hợp theo thuật toán Prophet

URI: /insert/prophet

Method: POST

4.5.5 API pruneProphetInsert

Chức năng: Tìm kiếm và chèn yêu cầu đi chung xe vào lịch trình hướng dẫn của taxi phù hợp theo thuật toán Prune Prophet

URI: /insert/prune-prophet

Method: POST

4.5.6 API scheduleOfDriver

Chức năng: Lấy ra lịch trình của một taxi

URI: /schedule/driver

Method: GET

4.5.7 API scheduleAllDriver

Chức năng: Lấy ra tất cả lịch trình của tất cả taxi đang hoạt động

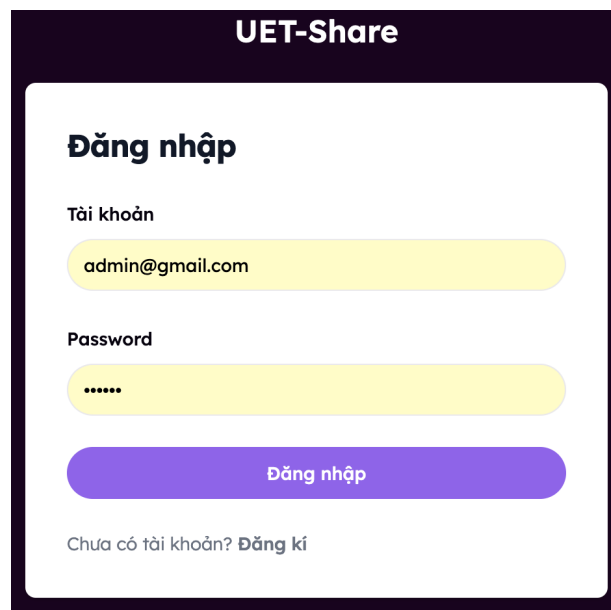
URI: /schedule/admin

Method: GET

4.6 Xây dựng giao diện website của ban quản lý

Dưới đây là giao diện dành cho ban quản lý tương tác và sử dụng để quản lý các taxi, tài xế cũng như theo dõi hoạt động của các taxi.

4.6.1 Đăng nhập

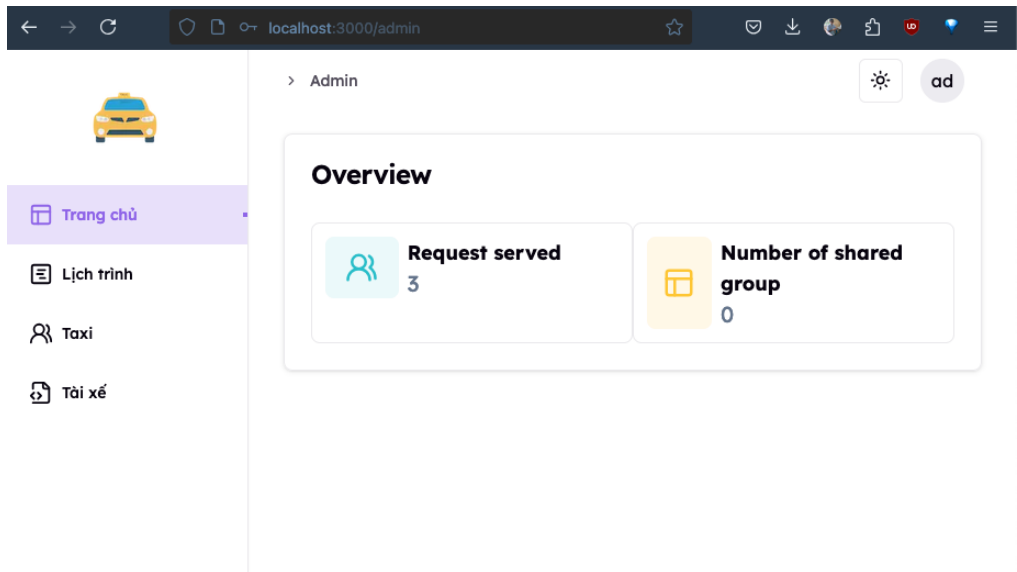


The image shows a login form titled "UET-Share" with the heading "Đăng nhập" (Login). It contains two input fields: "Tài khoản" (Account) with the value "admin@gmail.com" and "Password" with masked characters ".....". Below the fields is a blue "Đăng nhập" button. At the bottom, there is a link that says "Chưa có tài khoản? Đăng kí" (Don't have an account? Register).

Hình 4.9: Giao diện đăng nhập của ban quản lý

Đây là giao diện đăng nhập vào hệ thống của quản trị viên. Tài khoản của quản trị viên được cấp sẵn và không thể tự tạo tài khoản.

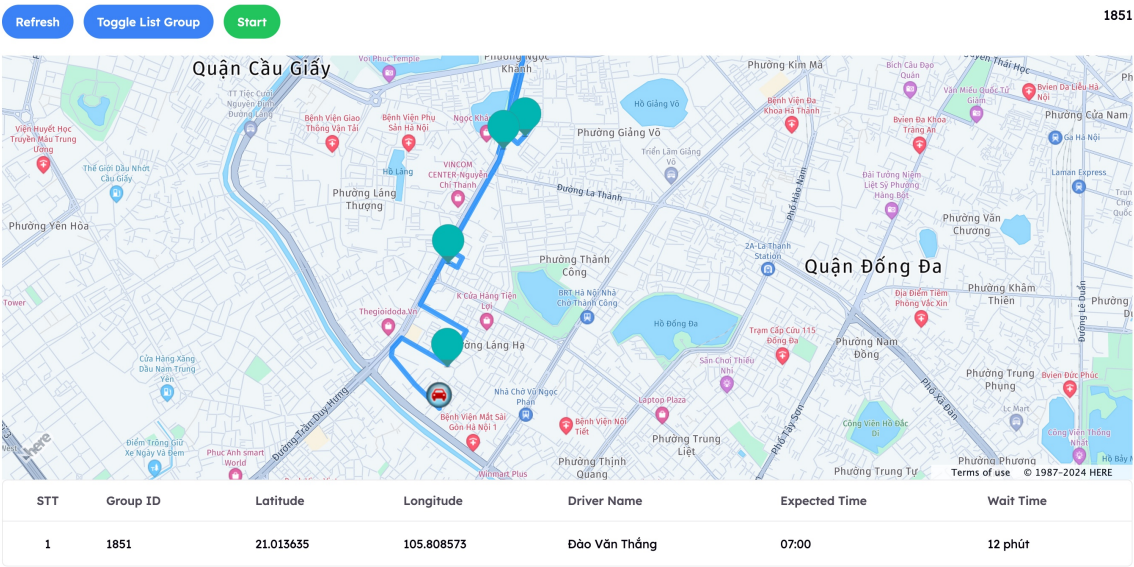
4.6.2 Giao diện chính



Hình 4.10: Giao diện quản lý chính

Đây là giao diện trang chủ khi ban quản lý đăng nhập vào hệ thống. Giao diện này hiển thị một vài thống kê của hệ thống như số yêu cầu đã được phục vụ, số xe chung đang di chuyển.

4.6.3 Xem thông tin chi tiết lịch trình hiện tại của các taxi đang hoạt động

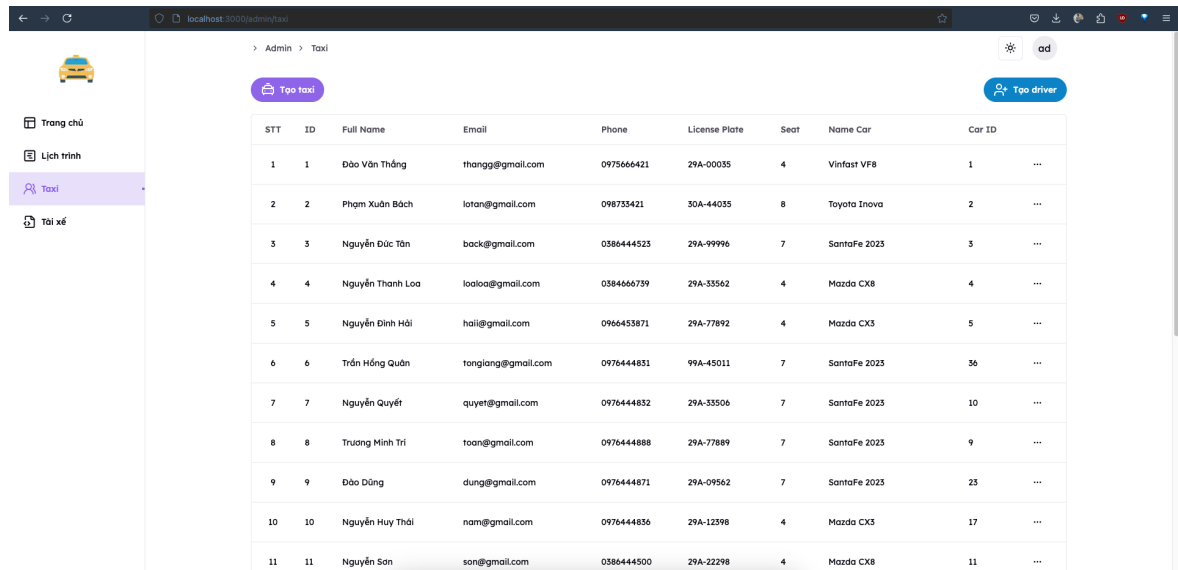


Hình 4.11: Giao diện nhóm các taxi và lịch trình hiện tại trên bản đồ

Từ danh sách các taxi, người dùng có thể chọn ra một xe đang hoạt động, hệ thống sẽ hiển thị thông tin chi tiết về lịch trình, vị trí hiện tại của taxi đó.

4.6.4 Tạo taxi mới

4.6.5 Danh sách taxi



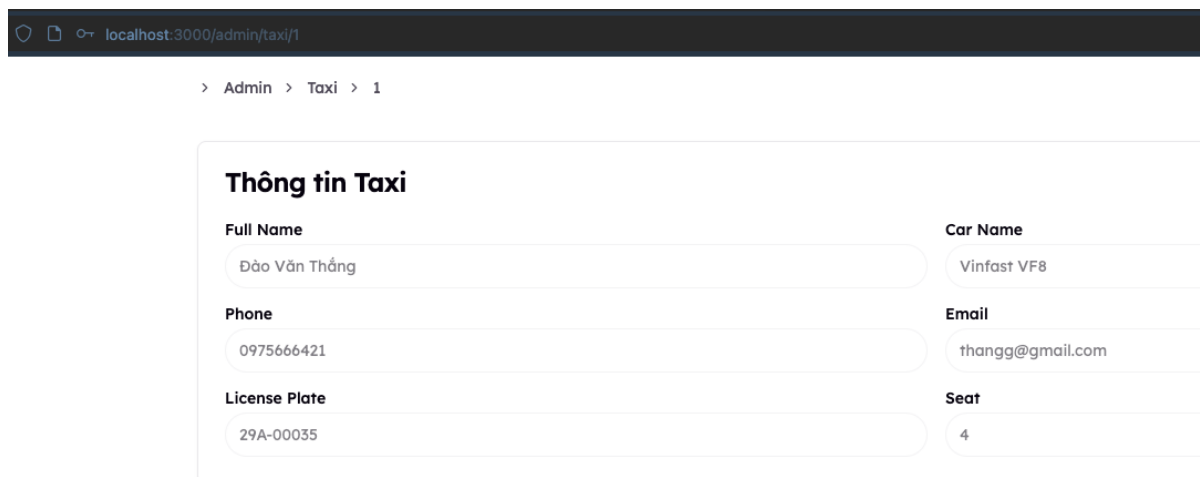
The screenshot shows a web application interface for managing taxis. On the left is a sidebar with navigation links: 'Trang chủ' (Home), 'Lịch trình' (Schedule), 'Taxi' (selected), and 'Tài xế' (Driver). The main content area is titled 'Admin > Taxi' and features a 'Tạo taxi' (Create taxi) button. Below this is a table listing 11 taxis. Each row contains columns for STT (Serial), ID, Full Name, Email, Phone, License Plate, Seat, Name Car, and Car ID. The table data is as follows:

STT	ID	Full Name	Email	Phone	License Plate	Seat	Name Car	Car ID
1	1	Đào Văn Thắng	thanggg@gmail.com	0975666421	29A-00035	4	Vinfast VF8	1
2	2	Phạm Xuân Bách	lotan@gmail.com	098735421	30A-44035	8	Toyota Inova	2
3	3	Nguyễn Đức Tân	back@gmail.com	0386444523	29A-99996	7	SantaFe 2023	3
4	4	Nguyễn Thanh Loa	loaloa@gmail.com	0384666739	29A-33562	4	Mazda CX8	4
5	5	Nguyễn Đình Hải	hail@gmail.com	0966453871	29A-77892	4	Mazda CX3	5
6	6	Trần Hồng Quân	tongiang@gmail.com	0976444831	99A-45011	7	SantaFe 2023	36
7	7	Nguyễn Quyết	quyet@gmail.com	0976444832	29A-33506	7	SantaFe 2023	10
8	8	Trương Minh Trí	toan@gmail.com	0976444888	29A-77889	7	SantaFe 2023	9
9	9	Đào Dũng	dung@gmail.com	0976444871	29A-09562	7	SantaFe 2023	23
10	10	Nguyễn Huy Thái	nam@gmail.com	0976444836	29A-12398	4	Mazda CX3	17
11	11	Nguyễn Sơn	son@gmail.com	0386444500	29A-22298	4	Mazda CX8	11

Hình 4.12: Giao diện danh sách taxi đang hoạt động

Ở giao diện trang chủ, người dùng nhấn vào phần "List taxi" hệ thống hiển thị giao diện danh sách các taxi đã được phân cho các tài xế.

4.6.6 Xem thông tin chi tiết thông tin của một taxi



The screenshot shows the 'Taxi Detail' form for taxi ID 1. The form is titled 'Thông tin Taxi' and contains two columns of input fields. The left column contains fields for 'Full Name', 'Phone', and 'License Plate'. The right column contains fields for 'Car Name', 'Email', and 'Seat'. The values entered in the fields are as follows:

Field	Value
Full Name	Đào Văn Thắng
Phone	0975666421
License Plate	29A-00035
Car Name	Vinfast VF8
Email	thanggg@gmail.com
Seat	4

Hình 4.13: Giao diện chi tiết taxi

Ở giao diện danh sách các taxi đang hoạt động ở phần 4.6.5. Người dùng có thể chọn một taxi, hệ thống sẽ hiển thị thông tin chi tiết về taxi và tài xế của taxi đó.

Tạo taxi

Tên xe
Toyota

Biển số xe
30A-12345

Số ghế
1

Create taxi

Hình 4.14: Giao diện tạo taxi mới

Ở phần 4.6.5, người dùng nhấn vào phần "Tạo Taxi" hệ thống hiển thị form để người dùng điền các thông tin về taxi mới và tạo.

4.6.7 Tạo tài khoản tài xế

Tạo tài xế

Email
taixe@gmail.com

Full Name
Pham Tai Xe

Password

Phone
1234567890

Car
▼

- 29A-29103 - Swift - 4
- 30A-23891 - Toyota Land Cruiser - 0
- 99A-0019 - SantaFe 2024 - 8

Create driver

Hình 4.15: Giao diện tạo tài khoản cho tài xế

Ở phần 4.6.5, người dùng nhấn vào phần "Tạo tài xế" hệ thống hiển thị form để người dùng điền các thông tin về tài khoản cho tài xế. Trong đó có một mục chứa danh sách các taxi của công ty mà chưa có tài xế nào phụ trách. Ban quản lý thực hiện chọn một trong các taxi đó đồng thời điền đầy đủ các thông tin khác và nhấn vào nút "Tạo" để tạo tài khoản cho tài xế.

Chương 5. KẾT LUẬN

5.1 Tổng kết

Sự gia tăng chóng mặt của phương tiện giao thông cá nhân, đặc biệt tại các đô thị lớn, đang gây áp lực nặng nề lên hạ tầng giao thông, dẫn đến tình trạng xuống cấp nhanh chóng và ô nhiễm môi trường ngày càng nghiêm trọng. Do đặc điểm tập trung của các yêu cầu di chuyển, việc dự đoán trước số lượng và chi tiết các yêu cầu đi xe có thể định tuyến được taxi về những nơi có nhu cầu lớn. Từ đó, mô hình có thể tận dụng được hết tiềm lực công nghệ cũng như dữ liệu từ các nghiên cứu gần đây, giúp các công ty cung cấp dịch vụ đi xe tối đa lợi nhuận. Đồng thời, góp phần giảm thiểu ô nhiễm môi trường và còn giúp tiết kiệm chi phí di chuyển so với việc sử dụng phương tiện cá nhân, từ đó thu hút đông đảo người dùng.

Khóa luận này đã đạt được một số kết quả đáng khích lệ, bao gồm việc xây dựng thành công hệ thống đi chung xe tận dụng được dữ liệu có sẵn về yêu cầu đặt xe, so sánh hiệu năng giữa các thuật toán khác nhau và đề xuất một thuật toán mới, đồng thời triển khai và ứng dụng thuật toán này trên bộ dữ liệu mô phỏng.

Tuy nhiên, do hạn chế về thời gian, kiến thức, cũng như nguồn lực thiết bị, hệ thống vẫn chưa hoàn thiện để có thể triển khai thực tế ngay lập tức. Tôi hy vọng rằng những ý tưởng và kết quả nghiên cứu của mình sẽ đóng góp một phần vào sự phát triển của dịch vụ đi chung xe, hướng tới giải quyết các vấn đề giao thông, môi trường và kinh tế hiện nay.

Rất mong nhận được sự góp ý của quý thầy cô để hoàn thiện khóa luận này hơn nữa. Mọi ý kiến đóng góp đều là vô cùng quý báu đối với tôi.

Tôi xin cảm ơn!

5.2 Định hướng phát triển

Những chức năng và định hướng phát triển hệ thống trong tương lai:

- + Phát triển app cho tài xế và người dùng
- + Mở rộng bản đồ lưới để hệ thống sử dụng được ở nhiều thành phố hơn
- + Nâng cấp hệ thống chỉ đường trong giao diện lịch trình của tài xế
- + Nghiên cứu bài toán về việc tính toán chi phí đi chung
- + Nghiên cứu bài toán chia sẻ chuyến đi kết hợp các yếu tố giao thông, thời tiết

Tài liệu tham khảo

- [1] Hà Nội - Diện tích đất dành cho giao thông không theo kịp tốc độ gia tăng của phương tiện.
- [2] Nguyên nhân ôto Hà Nội tăng gần gấp đôi sau 5 năm đạt hơn 1 triệu xe.
- [3] Matthias Sweet. “Traffic Congestion’s Economic Impacts: Evidence from US Metropolitan Regions”. In: *Urban Studies* 51.10 (2014), pp. 2088–2110. eprint: <https://doi.org/10.1177/0042098013505883>.
- [4] Shuo Ma, Yu Zheng, and Ouri Wolfson. “T-share: A large-scale dynamic taxi ridesharing service”. In: (2013), pp. 410–421.
- [5] Song Hu et al. “Frequent pattern growth algorithm-based association rule mining method of public transport travel stability”. In: (2020).
- [6] Desheng Zhang et al. “CoRide: Carpool service with a win-win fare model for large-scale taxicab networks”. In: (2013).
- [7] Xu Geng et al. “Spatiotemporal multi-graph convolution network for ride-hailing demand forecasting”. In: *Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence and Thirty-First Innovative Applications of Artificial Intelligence Conference and Ninth AAAI Symposium on Educational Advances in Artificial Intelligence*. AAAI’19/IAAI’19/EAAI’19. Honolulu, Hawaii, USA: AAAI Press, 2019. ISBN: 978-1-57735-809-1.
- [8] Ren Ozeki et al. “One Model Fits All: Cross-Region Taxi-Demand Forecasting”. In: Dec. 2023, pp. 1–4.
- [9] Yongxin Tong et al. “A Unified Approach to Route Planning for Shared Mobility”. In: (2018).
- [10] Yongxin Tong et al. “Unified Route Planning for Shared Mobility: An Insertion-based Framework”. In: *ACM Trans. Database Syst.* 47.1 (May 2022). ISSN: 0362-5915.
- [11] Wilson NHM Weissberg R W Hauser J. “ADVANCED DIAL-A-RIDE ALGORITHMS RESEARCH PROJECT”. In: (1976).
- [12] Anupam Gupta et al. “Dial a Ride from k-forest”. In: *ACM Trans. Algorithms* 6.2 (Apr. 2010). ISSN: 1549-6325.

- [13] Yuxiang Zeng, Yongxin Tong, and Lei Chen. “Last-mile delivery made practical: an efficient route planning framework with theoretical guarantees”. In: *Proc. VLDB Endow.* 13.3 (Nov. 2019), 320–333. ISSN: 2150-8097.
- [14] Esteban Feuerstein and Leen Stougie. “On-line single-server dial-a-ride problems”. In: *Theoretical Computer Science* 268.1 (2001). On-line Algorithms ’98, pp. 91–105. ISSN: 0304-3975.
- [15] Song Hu et al. “Frequent-pattern Growth Algorithm Based Association Rule Mining Method of Public Transport Travel Stability”. In: *International Journal of Sustainable Transportation* 15 (Sept. 2020).
- [16] Van Tran and Thi Vu. “A New Method for Forming Rideshare Groups”. In: Oct. 2023, pp. 1–6.
- [17] Niels Agatz et al. “Optimization for dynamic ride-sharing: A review”. In: *European Journal of Operational Research* 223.2 (2012), pp. 295–303. ISSN: 0377-2217.
- [18] Yan Huang et al. “Large scale real-time ridesharing with service guarantee on road networks”. In: *Proc. VLDB Endow.* 7.14 (Oct. 2014), 2017–2028. ISSN: 2150-8097.
- [19] Công Thành Võ. “Nghiên cứu tối ưu lập lịch cho bài toán đi chung”. In: (2023).

Phụ lục

Chương A. CHI TIẾT CÁC BẢNG Ở CƠ SỞ DỮ LIỆU

Dưới đây là mô tả chi tiết các bảng có trong cơ sở dữ liệu.

Bảng A.1: Bảng admin

Tên trường	Mô tả	Kiểu dữ liệu
id (PK)	Khóa chính, tự động tăng	INT
email	Email của tài khoản	VARCHAR
password	Mật khẩu tài khoản đã được mã hóa	VARCHAR

Bảng A.2: Bảng passenger

Tên trường	Mô tả	Kiểu dữ liệu
id (PK)	Khóa chính, tự động tăng	INT
email	Email của tài khoản	VARCHAR
full_name	Họ tên đầy đủ của hành khách	VARCHAR
password	Mật khẩu tài khoản đã được mã hóa	VARCHAR
phone	Số điện thoại của hành khách	VARCHAR

Bảng A.3: Bảng driver

Tên trường	Mô tả	Kiểu dữ liệu
id (PK)	Khóa chính, tự động tăng	INT
email	Email của tài khoản	VARCHAR
full_name	Họ tên đầy đủ của tài xế	VARCHAR
license_plate	Biển số xe của taxi mà tài xế được phân bởi công ty	VARCHAR
name_car	Tên xe taxi mà tài xế được phân bởi công ty	VARCHAR
password	Mật khẩu tài khoản đã được mã hóa	VARCHAR
phone	Số điện thoại của tài xế	VARCHAR
seat	Dung lượng chỗ ngồi của taxi	INT
taxi_id	Khóa ngoại trở đến trường id của bảng taxi	INT

Bảng A.4: Bảng taxi

Tên trường	Mô tả	Kiểu dữ liệu
id (PK)	Khóa chính, tự động tăng	INT
license_plate	Biển số xe của taxi	VARCHAR
name_car	Tên xe taxi	VARCHAR
seat	Dung lượng chỗ ngồi của taxi	INT

Bảng A.5: Bảng OTP

Tên trường	Mô tả	Kiểu dữ liệu
email (PK)	Khóa ngoại trở đến trường email của các bảng admin, passenger, driver	VARCHAR
otp	Mã gồm 6 chữ số	INT

Bảng A.6: Bảng route

Tên trường	Mô tả	Kiểu dữ liệu
id (PK)	Khóa chính, tự động tăng, id của tuyến đường thường xuyên	INT
account_id	Khóa ngoại trỏ đến trường id của các bảng admin, passenger, driver	INT
address_start	Tên vị trí bắt đầu tuyến đường thường xuyên	VARCHAR
address_end	Tên vị trí kết thúc tuyến đường thường xuyên	VARCHAR
cost	Chi phí của tuyến đường nếu ghép xe đi chung	DOUBLE
group_id	Khóa ngoại trỏ đến trường id của bảng group_id	INT
lat_end	Vĩ độ kết thúc của tuyến đường	DOUBLE
lat_start	Vĩ độ bắt đầu của tuyến đường	DOUBLE
length_route	Độ dài tuyến đường	DOUBLE
lng_start	Kinh độ bắt đầu của tuyến đường	DOUBLE
lng_end	Kinh độ kết thúc của tuyến đường	DOUBLE
time_start	Thời gian bắt đầu tuyến đường	DOUBLE
time_end	Thời gian kết thúc tuyến đường	DOUBLE

Bảng A.7: Bảng grid

Tên trường	Mô tả	Kiểu dữ liệu
id (PK)	Khóa chính, tự động tăng	INT
anchor_lat	Kinh độ điểm neo của ô lưới	DOUBLE
anchor_lng	Vĩ độ điểm neo của ô lưới	DOUBLE
lat_start	Kinh độ bắt đầu của ô lưới	DOUBLE
lat_end	Kinh độ kết thúc của ô lưới	DOUBLE
lng_start	Vĩ độ bắt đầu của ô lưới	DOUBLE
lng_end	Vĩ độ kết thúc của ô lưới	DOUBLE

Bảng A.8: Bảng edge

Tên trường	Mô tả	Kiểu dữ liệu
origin (PK)	Khóa ngoại trỏ đến trường id của bảng grid thể hiện id của ô lưới của điểm bắt đầu	INT
destination (PK)	Khóa ngoại trỏ đến trường id của bảng grid thể hiện id của ô lưới của điểm kết thúc	INT
distance	Khoảng cách di chuyển từ ô lưới origin đến ô lưới destination	DOUBLE
duration	Thời gian di chuyển từ ô lưới origin đến ô lưới destination	DOUBLE

Bảng A.9: Bảng request_ride

Tên trường	Mô tả	Kiểu dữ liệu
id (PK)	Khóa chính, tự động tăng	INT
address_start	Tên địa điểm đón	VARCHAR
address_end	Tên địa điểm đến	VARCHAR
capacity	Số lượng người tham gia	INT
cost	Giá cước	DOUBLE
group_id	Khóa ngoại trỏ đến trường id của bảng group_frequent	INT
lat_destination	Kinh độ điểm đến	DOUBLE
lat_origin	Kinh độ điểm đón	DOUBLE
lng_destination	Vĩ độ điểm đến	DOUBLE
lng_origin	Vĩ độ điểm đón	DOUBLE
passenger_id	Khóa ngoại trỏ đến trường id của bảng passenger thể hiện khách hàng tạo yêu cầu	INT
pick_up_time	Thời gian bắt đầu	DOUBLE
status_id	Khóa ngoại trỏ đến trường id của bảng type_status	INT
booking_date	Thời gian kiểu ngày/ tháng/ năm yêu cầu được tạo	DATE

Bảng A.10: Bảng predicted_request

Tên trường	Mô tả	Kiểu dữ liệu
id (PK)	Khóa chính, tự động tăng	INT
address_start	Tên địa điểm đón	VARCHAR
address_end	Tên địa điểm đến	VARCHAR
capacity	Số lượng người tham gia	INT
cost	Giá cước	DOUBLE
lat_destination	Kinh độ điểm đến	DOUBLE
lat_origin	Kinh độ điểm đón	DOUBLE
lng_destination	Vĩ độ điểm đến	DOUBLE
lng_origin	Vĩ độ điểm đón	DOUBLE
pick_up_time	Thời gian bắt đầu	DOUBLE
show_time	Thời gian xuất hiện	DOUBLE
booking_date	Thời gian kiểu ngày/ tháng/ năm yêu cầu được tạo	DATE

Bảng A.11: Bảng type_status

Tên trường	Mô tả	Kiểu dữ liệu
id (PK)	Khóa chính, tự động tăng	INT
type	Trạng thái của yêu cầu đặt xe gồm: serving: yêu cầu đặt xe đã được lên lịch và đang thực hiện done: chuyến đi của khách hàng đã thực hiện xong cancel: yêu cầu đã được khách hàng hủy	VARCHAR

Bảng A.12: Bảng group_frequent

Tên trường	Mô tả	Kiểu dữ liệu
id (PK)	Khóa chính, tự động tăng	INT
driver_id	Khóa ngoại trỏ đến trường id của bảng driver thể hiện tài xế nào thực hiện chuyến đi của group này	INT
is_prophet	Giá trị thể hiện taxi đi chung	INT

Bảng A.13: Bảng schedule

Tên trường	Mô tả	Kiểu dữ liệu
group_id (PK)	Khóa ngoại trỏ đến trường id của bảng group_frequent	INT
passenger_id (PK)	Khóa ngoại trỏ đến trường id của bảng passenger	INT
capacity_available	Dung lượng chỗ ngồi còn trống của taxi ở điểm hiện tại	INT
expected_time	Thời gian dự kiến taxi đến điểm hiện tại	DOUBLE
lat	Kinh độ	DOUBLE
lng	Vĩ độ	DOUBLE
location_id	Khóa ngoại trỏ đến trường id của bảng type_location	INT
schedule_id	Khóa ngoại trỏ đến trường id của bảng type_schedule	INT
late_time	Thời gian muộn nhất taxi cần đến điểm này	DOUBLE

Bảng A.14: Bảng *guidance_schedule*

Tên trường	Mô tả	Kiểu dữ liệu
group_id (PK)	Khóa ngoại trở đến trường id của bảng group_frequent	INT
passenger_id (PK)	Khóa ngoại trở đến trường id của bảng passenger	INT
capacity_available	Dung lượng chỗ ngồi còn trống của taxi ở điểm hiện tại	INT
expected_time	Thời gian dự kiến taxi đến điểm hiện tại	DOUBLE
lat	Kinh độ	DOUBLE
lng	Vĩ độ	DOUBLE
location_id	Khóa ngoại trở đến trường id của bảng type_location	INT
schedule_id	Khóa ngoại trở đến trường id của bảng type_schedule	INT
time_late	Thời gian muộn nhất taxi cần đến điểm này	DOUBLE
wait_time	Thời gian đợi của taxi tại địa điểm hiện tại	DOUBLE

Bảng A.15: Bảng *type_location*

Tên trường	Mô tả	Kiểu dữ liệu
id (PK)	Khóa chính, tự động tăng	INT
type	Loại điểm đến gồm: origin: điểm đón destination: điểm trả	VARCHAR

Bảng A.16: Bảng type_schedule

Tên trường	Mô tả	Kiểu dữ liệu
id (PK)	Khóa chính, tự động tăng	INT
type	Loại schedule gồm: online: yêu cầu đi chung xe trực tuyến predicted: yêu cầu đi chung xe dự đoán	VARCHAR